

基于 SIMD 的 ANN 多线程与加速器并行优化

Pthread、OpenMP CPU/Target、HNSW 与跨平台性能分析

2026 年 5 月 22 日

姓名： 匡航逸 学号： 2412235
专业： 计算机科学与技术 数据集： DEEP100K
评价指标： recall@100，并在 $\text{recall@100} \geq 0.95$ 下最小化平均查询延迟
GitHub： <https://github.com/xzxxntxdy/Parallel-programming-NKU>

摘要

本报告在 SIMD 阶段 Flat、SQ8、PQ、SDC 与 FastScan 距离计算优化的基础上，继续研究近似最近邻搜索（Approximate Nearest Neighbor Search, ANNS）的多线程与加速器并行化。实验统一采用 DEEP100K 数据集，并在 $\text{recall@100} \geq 0.95$ 的候选中选择最低查询延迟。围绕这一目标，本文实现并分析了 Flat 精确搜索的 Pthread 与 OpenMP CPU 并行、SQ8/PQ/SDC/FastScan 的量化查表优化、IVF/IVF-PQ 的倒排裁剪、HNSW 图索引及其查询内并行变体，以及 Intel GPU 上的 OpenMP target batch IVF 和 oneAPI/SYCL 对比。x86 平台最终最优为 HNSW query-level 并行：StdAsync, ef=64, threads=32，全量查询平均延迟 $9.316 \mu\text{s}/\text{query}$ ， $\text{recall@100}=0.952683$ ；ARM/Kunpeng 平台最终最优为 StdThread, ef=64, threads=16，全量 test.sh 结果为 $70.8786 \mu\text{s}/\text{query}$ ， $\text{recall@100}=0.952643$ 。OpenMP CPU 在 x86 和 ARM 上覆盖 Flat、base-split、PQ-ADC 与 IVF sweep，最优均为 IVF dynamic schedule：x86 为 $0.082060 \text{ ms}/\text{query}$ ， $\text{recall@100}=0.956950$ ；ARM 为 $0.338880 \text{ ms}/\text{query}$ ， $\text{recall@100}=0.950767$ 。OpenMP target 在 Intel GPU 上执行 batch IVF 设备端候选扫描，最优配置为 nprobe=81, candidate cap=3840，延迟 $0.186502 \text{ ms}/\text{query}$ ， $\text{recall@100}=0.950170$ 。实验表明：多线程的收益取决于算法瓶颈，HNSW 的图结构剪枝远比简单扩大线程数有效；OpenMP CPU 便于表达循环并行但调度开销高于手写 Pthread 路径；GPU 路径在规则候选扫描上有吞吐优势，但需要更强的候选裁剪和设备端 top-k 才能进一步接近 CPU HNSW 的端到端延迟。

目录

1 问题定义与评价指标	3
1.1 kNN 与 ANNS 问题	3
1.2 recall@100 与最优选择规则	3
1.3 实验平台	3
2 实验矩阵与消融设计	3
3 并行开销模型	5
4 总体优化路线	5
5 Flat-SIMD 与 Pthread 并行	6
5.1 算法设计	6
5.2 优化过程与实验结果	7
6 SQ8、PQ、SDC 与 FastScan	8
6.1 SQ8：标量量化与 rerank	8

6.2	PQ-ADC 与 PQ-SDC	8
6.3	FastScan 与查表布局	9
6.4	量化路径的优化复盘	9
7	IVF 与 IVF-PQ	10
7.1	倒排索引结构	10
7.2	nprobe 的 trade-off	11
7.3	IVF-PQ 与局部 PQ	11
8	HNSW 图索引	12
8.1	HNSW 查询过程	12
8.2	query-level 并行	12
8.3	查询内并行探索	13
9	OpenMP CPU 并行化	14
9.1	OpenMP host CPU 的实现范围	14
9.2	schedule、划分方式与一致性	15
9.3	OpenMP CPU 实验结果	15
10	OpenMP Target 与 oneAPI/SYCL	16
10.1	加速器实现目标	16
10.2	OpenMP target IVF	16
10.3	与 oneAPI/SYCL 的比较	17
11	x86 与 ARM/Kunpeng 跨平台分析	18
11.1	算法最优点是否迁移	18
11.2	跨平台性能差异的来源	19
11.3	构建时间与查询延迟	19
11.4	最终参数选择依据	20
12	Profiling 与体系结构相关优化	21
12.1	阶段耗时 profiling	21
12.2	cache 与内存访问分析	21
12.3	复杂性与线程管理代价	22
13	综合讨论：算法结构与硬件匹配	22
13.1	从计算密集到访存密集	22
13.2	线程粒度的选择原则	22
13.3	近似误差与参数敏感性	23
13.4	跨平台差异的解释	23
13.5	对最终方案的工程判断	23
14	实现组织与结果选择流程	24
14.1	可靠性检查与平台迁移	24
15	优化过程总结	25
16	结论	25
	生成式 AI 辅助学习附录	26
	参考文献	31

1 问题定义与评价指标

1.1 kNN 与 ANNS 问题

给定 base 集合 $X = \{x_i\}_{i=1}^N$ 、查询向量 q 和距离函数 $d(\cdot, \cdot)$ ，精确 k 近邻搜索的目标是返回距离最小的 k 个向量：

$$\text{kNN}(q) = \arg \min_{S \subset X, |S|=k} \sum_{x \in S} d(q, x).$$

在 DEEP100K 上， $N = 100000$ ，向量维度 $d = 96$ ，ground truth 提供 top-100。因此本文的目标输出为每个 query 的 top-100 候选。精确 Flat 搜索需要访问全部 base 向量，计算复杂度为 $O(Nd)$ ；ANNS 则通过量化、倒排索引或图索引减少候选访问量，以可控的 recall 换取更低 latency。

本文使用的距离形式与 SIMD 阶段一致。每个候选向量与 query 的相似度或距离由 SIMD 核进行批量计算。Pthread 阶段的新增问题是：当单次距离计算已经被 SIMD 压缩之后，继续增加线程是否仍然有效？如果有效，线程应该并行 query、base、倒排链、PQ 查表，还是图搜索入口？这些问题决定了后续算法设计。

1.2 recall@100 与最优选择规则

设算法返回集合为 R_q ，ground truth 集合为 G_q ，两者大小均为 100，则单个 query 的召回率为

$$\text{recall@100}(q) = \frac{|R_q \cap G_q|}{100}.$$

本文统计的 recall 为所有 query 的平均值，并采用如下约束优化准则：

$$\theta^* = \arg \min_{\theta \in \Theta} \text{Latency}(\theta) \quad \text{s.t.} \quad \text{recall@100}(\theta) \geq 0.95.$$

其中 θ 包括算法、线程数、候选数 p 、倒排簇数 nlist 、搜索簇数 nprobe 、HNSW 的 ef 、GPU candidate cap 等参数。后续所有表格和图中的“最优”均按这一规则筛选。

1.3 实验平台

实验覆盖三个层次：x86 Windows 本地 CPU、ARM/Kunpeng CPU，以及 x86 机器上的 Intel GPU 加速器。x86 CPU sweep 使用 1000 个 query 进行完整消融；ARM/Kunpeng 使用 quick sweep 选择参数，并通过全量 `test.sh` 验证最终路径；GPU 实验采用 `batch=1000` 的 IVF 设备端扫描。

表 1: 实验平台与评价口径

平台	硬件与后端	运行规模	作用
x86 Windows	Intel Core i9-14900HX, AVX2, MSVC / Intel DPC++ 2024.2.1	CPU sweep 1000 queries; final 全量 queries	本地完整消融、Pthread/OpenMP CPU、HNSW、OpenMP target 与 SYCL 对比。
ARM/Kunpeng	HiSilicon Kunpeng-920, aarch64, NEON, OpenEuler 5.10	quick sweep 500 queries; final 全量 queries	Pthread/OpenMP CPU 跨平台分析与 ARM 最终运行路径。
Intel GPU	Intel RaptorLake-S Mobile Graphics Controller, Level Zero / OpenCL	batch=1000	OpenMP target 与 oneAPI/SYCL 加速器实验。

2 实验矩阵与消融设计

ANN 多线程优化容易出现“单点最优不可解释”的问题：某个配置可能在一次运行中很快，但无法说明它是算法结构收益、线程调度收益，还是 recall 降低带来的假象。本文因此采用分层消融矩阵。第一层固定算法族，扫描线程数，观察并行扩展；第二层固定线程数，扫描候选规模或搜索宽度，观察 recall-latency 曲线；第三层比较不同平台上的最优点，判断参数是否具有迁移性。

实验矩阵不是为了穷举所有可能参数，而是围绕每类算法的关键控制量展开。Flat 的关键量是线程数和局部 top- p ；SQ8/PQ/FastScan 的关键量是量化粒度、候选数 p 和查表布局；IVF 的关键量是

nlist 与 **nprobe**；HNSW 的关键量是 **ef**、线程工具和查询内并行策略；GPU IVF 的关键量是 **batch**、**nprobe**、**candidate cap**、距离矩阵回传规模与 **top-k selection** 开销。

实验矩阵按 CPU 多线程、图索引查询策略和异构加速三条线组织。CPU 线同时包含 Pthread 和 OpenMP host：Pthread 路径覆盖 Flat、SQ8、PQ、SDC/FastScan、IVF、IVF-PQ 和 HNSW；OpenMP CPU 路径覆盖 Flat query-level、Flat base-split、PQ-ADC 和 IVF，并比较 static/dynamic schedule。图索引线比较标准 HNSW query-level 搜索、Layer 0 边划分、点划分、多入口点并行和 IVF+HNSW。异构线使用 OpenMP target、oneAPI/SYCL 和 OpenCL/CUDA 参照路径分析 GPU batch IVF 的收益与瓶颈。ARM/Kunpeng 结果采用 quick sweep 选型并用 **test.sh** 全量验证最终路径，所有实测 CSV 都回填到同一报告体系中。

表 2: 主要消融维度与预期观察

算法族	消融参数	主要观测指标	预期 trade-off
Flat	SIMD kernel、线程数、local top- <i>p</i>	latency、recall、线程扩展	精确但扫描全量；base-split 需要 top-k merge。
SQ8	候选数 <i>p</i> 、rerank 规模、线程数	近似筛选时间、精排时间	<i>p</i> 增大提升 recall，但 rerank 抬高延迟。
PQ/SDC	子空间数 <i>M</i> 、候选数 <i>p</i> 、ADC/SDC	LUT 构建、查表累加、recall	查表降低计算量，但随机访存更明显。
FastScan	block 大小、候选数 <i>p</i> 、线程数	SIMD 查表吞吐、cache 行为	layout 越适合 SIMD，单候选越快；候选规模仍受限制。
IVF	nlist 、 nprobe 、线程数	coarse 时间、scan 时间、recall	nprobe 是 recall 与扫描量的直接旋钮。
HNSW	ef 、线程数、查询内并行策略	graph visit 数、队列维护、recall	query-level 稳定；查询内切分可能破坏剪枝。
OpenMP CPU	schedule、线程数、局部 top- <i>p</i> 、 nprobe	host CPU latency、schedule 开销、recall	dynamic 缓解负载不均，但运行时调度成本高于手写固定分配。
OpenMP target	nprobe 、 candidate cap 、 batch	GPU scan、selection、传输/同步	需要足够 batch 和候选裁剪， selection 下沉与回传规模是后续关键。

所有消融结果都保留构建时间和查询时间的区分。构建时间不进入单 query latency，但构建代价会影响算法选择。例如 Flat 没有构建时间，但查询慢；HNSW 构建时间更高，但查询显著更快。因此本文在最终分析中同时报告 build time 和 query latency。

为了避免 recall 更低的配置被误判为“优化成功”，每个实验族都先过滤 $\text{recall}@100 \geq 0.95$ ，再按 latency 排序。没有达到 0.95 的点仍然出现在 trade-off 图中，但不会进入最优表。HNSW 查询内并行、IVF-HNSW 和 GPU batch IVF 都按同一规则评价，保证不同并行工具之间的比较口径一致。

问题规模在本实验中不只指原始数据集大小。DEEP100K 固定了 $N = 100000$ 和 $D = 96$ ，而不同算法真正改变的是每个 query 的有效工作量：Flat 的候选规模为 N ，PQ/FastScan 的候选规模由 p 控制，IVF 由 **nprobe** 控制实际倒排链扫描量，HNSW 由 **ef** 控制图访问宽度，GPU IVF 还受到 **candidate cap** 和 **batch size** 的共同约束。因此“不同问题规模”的实验体现在三组 sweep 中：线程规模 $T = 1, 2, 4, 8, 16, 32$ ，候选规模 $p/\text{nprobe}/\text{ef}$ ，以及 query batch 规模。表 3 直接列出单线程与最佳并行点的差异，后续章节再解释每类算法为什么扩展性不同。

表 3: 单线程与最佳并行点对照

平台	路径	规模参数	单线程 latency	最佳并行配置	并行 latency	相对 1T	recall@100	解释
x86	Pthread Flat-SSE	$N = 100K$	2.079648 ms	$T = 32$	0.130451 ms	$15.94\times$	1.000000	计算量大，线程扩展明显但仍扫描全量。
x86	Pthread IVF	nprobe=32	0.685096 ms	$T = 32$	0.044471 ms	$15.41\times$	0.956950	候选已裁剪，仍能利用多线程扫描倒排链。
x86	HNSW StdAsync	ef=64	0.078811 ms	$T = 32$	0.009739 ms	$8.09\times$	0.954540	单 query 工作量大，线程调度占比上升。
x86	OpenMP CPU IVF	nprobe=32	0.569579 ms	dynamic $T = 32$	0.082060 ms	$6.94\times$	0.956950	runtime 调度换取倒排链负载均衡。
ARM	Pthread Flat base-split	$p = 100$	6.525818 ms	$T = 16$	0.717344 ms	$9.10\times$	1.000000	16 线程后受内存带宽和归并限制。
ARM	Pthread IVF	nprobe=32	1.045134 ms	$T = 32$	0.193230 ms	$5.41\times$	0.956240	候选规模相同，NEON 与内存层次决定绝对延迟。
ARM	HNSW StdThread	ef=64	0.580118 ms	$T = 16$	0.085378 ms	$6.80\times$	0.954280	16 线程最稳，32 线程出现调度收益下降。
ARM	OpenMP CPU IVF	nprobe=32	1.561803 ms	dynamic $T = 8$	0.338880 ms	$4.61\times$	0.950767	quick sweep 中较少线程即可跨过目标线。

表 3 中 Flat 的并行加速比最高，但最终 latency 不是最低，说明多线程只是在固定候选集合上降低常数。HNSW 的相对加速比小一些，却有最低端到端延迟，因为图导航先减少了候选访问量。OpenMP CPU IVF 在两个平台上都比 Pthread IVF 慢，不能简单归因于编译器；它反映的是 OpenMP dynamic schedule、隐式 barrier 和 chunk 管理在短查询上的固定成本。这个对照是后文所有 trade-off 分析的基准：评价并行策略时要同时看候选规模、线程规模和同步/归并成本。

3 并行开销模型

多线程优化的实际收益可以用一个简化模型理解。对某个算法配置，平均查询延迟可以拆成

$$T(P) = \frac{T_{\text{work}}}{S(P)} + T_{\text{sync}}(P) + T_{\text{merge}}(P) + T_{\text{mem}}(P),$$

其中 P 为线程数， $S(P)$ 是有效并行加速比。理想情况下 $S(P) \approx P$ ，但 ANN 查询很少达到理想线性加速，因为 top-k merge、随机访存、cache miss 和线程调度都会随 P 增长。

从任务划分角度看，ANN 可以对应到传统矩阵并行中的“水平划分”和“垂直划分”。水平划分把 query、base 候选、倒排链或图分区切给不同线程；垂直划分把维度、PQ 子空间、LUT 构建或距离累加拆开。二者的一致性代价不同：水平划分通常需要 top-k 归并，垂直划分需要距离部分和归约，图搜索划分还要维护全局候选队列的语义。

表 4: ANN 中不同任务划分的复杂性与一致性

划分方式	对应实现	主要复杂度	一致性保证	主要风险
query 水平划分	Flat/PQ/IVF/HNSW query-level	$O(Q \cdot C_q/T)$	每个 query 的 heap、LUT、visited 私有；结果槽按 query id 写入	需要足够 query batch；不能降低单 query 延迟。
base/candidate 水平划分	Flat base-split、IVF 倒排链分片	$O(C_q D/T) + O(Tp \log(Tp))$	每线程局部 top-p，并行后全局归并	p 太小影响 recall，p 太大增加 merge。
维度/子空间垂直划分	PQ LUT、ADC 子空间累加	$O(MK_s D_s/T)$ 或 $O(CM/T)$	部分距离求和后才能更新 top-k	需要额外 reduction；LUT 阶段占比小时收益有限。
分区/分片划分	IVF+HNSW、多入口图搜索	$O(\sum_i V_i(e_f)) + O(Tk \log Tk)$	子图局部 top-k 最后统一去重归并	两层近似误差叠加，重复访问增加。
设备 batch 二维划分	OpenMP target/SYCL IVF	$O(B \cdot cap \cdot D/G)$	候选 id 矩阵只读，距离矩阵回传后 host selection	无效 slot、回传规模和 host-device 同步。

表 4 解释了为什么最终更偏向 query-level 和规则 batch scan。query-level 的一致性最简单，线程之间不共享可变搜索状态；base-split 和倒排链分片可以降低单 query 扫描时间，但必须付出 top-k reduction；PQ 子空间并行看起来规则，但 LUT 构建只占较小比例，拆分后还需要 barrier 再进入查表扫描；HNSW 查询内划分的难点最大，因为局部线程很难同时保持全局候选队列的剪枝语义。

Flat 的 T_{work} 最大，且每个候选距离计算相对规则，因此它最容易从线程数增加中获益。但 Flat 的 T_{mem} 也很大，因为每个 query 都要读取全部 base。PQ/FastScan 降低了 T_{work} ，使查表和内存访问变成主导；此时继续增加线程数，收益会比 Flat 更早饱和。IVF 同时降低 T_{work} 和 T_{mem} ，但会引入 coarse selection 和倒排链长度不均衡。HNSW 进一步降低访问候选数量，但其队列依赖导致单 query 内并行的 T_{sync} 和 T_{merge} 很高。

GPU 路径的模型略有不同。batch IVF 的设备端吞吐可以很高，但如果候选列表访问不连续、局部 top-k 选择复杂， T_{mem} 和 T_{merge} 仍会成为瓶颈。OpenMP target 和 SYCL 的实验都表明，GPU 优化不能只看并行线程数量，还要看候选裁剪是否足够强、selection 是否足够轻、batch 是否足够大。

这个模型解释了后文的主要结果：Flat 线程扩展明显但最终不快；FastScan 单候选很快但候选量仍偏大；IVF 在非图索引中最稳定；HNSW 由于候选访问量最低，最终在 CPU 上胜出；OpenMP target 能超过部分 CPU 非图路径，但无法超过 HNSW。

4 总体优化路线

本阶段的优化路线可以概括为“先压缩单次计算，再减少候选数量，最后选择合适并行粒度”。SIMD 阶段已经解决了单次距离计算的向量化问题；Pthread 阶段进一步面对的是调度、同步、内存访问和索引结构。图 1 给出了所有核心路径在 $\text{recall}@100 \geq 0.95$ 下的最低延迟，后续章节将逐一解释这些结果是如何得到的。

代码结构也按这一路线重构。距离计算核保持无状态，只接收 query 指针、base 指针和维度参数；索引层只负责生成候选集合，例如 Flat 的连续区间、IVF 的倒排链、PQ 的压缩 code 或 HNSW 的邻接点。CPU 多线程层采用 Pthread-style worker 模型：每个 worker 接收只读索引指针、query 步长、线程编号、线程私有 heap 和结果写入区，热循环中不持有锁、不共享 top-k 结构；OpenMP target 层则把候选矩阵、base 向量和距离输出展平成设备 kernel 可直接遍历的数组。这样可以避免把线程状态写进 SIMD 核，也使同一套 AVX2/NEON 距离函数能够复用于 Flat、IVF rerank、HNSW 节点距离和量化候选精排。

这一拆分还统一了后续实验的记录方式。每个算法族的 CSV 都保存算法名、线程数、核心参数、build time、平均 latency 和 $\text{recall}@100$ ；最终选点由脚本按 $\text{recall}@100 \geq 0.95$ 过滤后排序。图表不是手工录入结果，而是从这些结构化记录中重新绘制。也就是说，Pthread 线程策略、HNSW 查询内并行变体、OpenMP target 的 `nprobe/candidate cap` 调参和 SYCL 对比，最终都落到同一评价口径下。

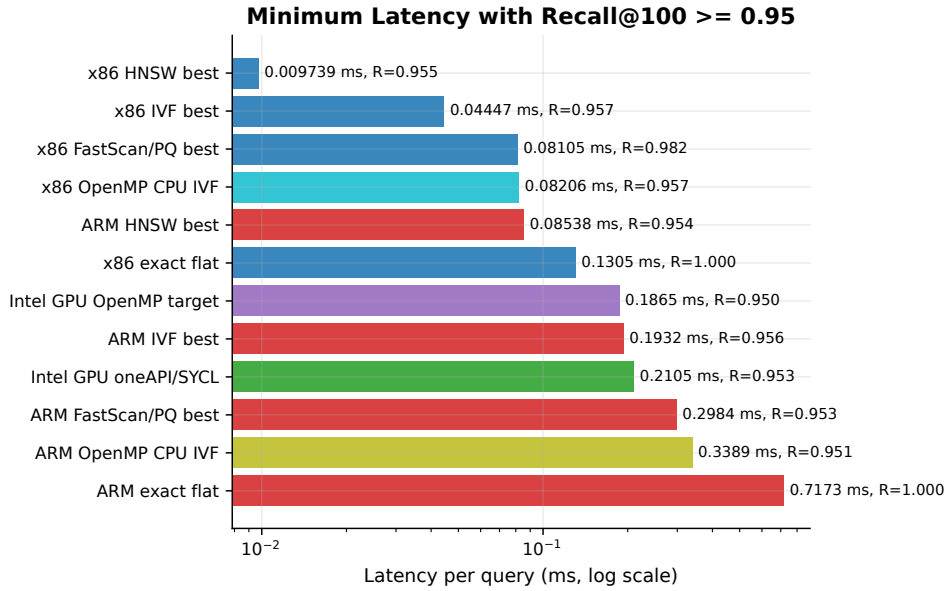


图 1: 核心算法在 recall@100 $\geq$ 0.95 下的最低延迟。横轴为平均查询延迟，颜色区分 x86、ARM 与 Intel GPU 后端。

从总体结果看，HNSW 图索引在 x86 和 ARM 上均取得最终最低延迟；IVF 是最稳定的非图索引；PQ/FastScan 通过量化和查表降低单候选成本，但在召回率目标下仍需较多候选；OpenMP target 与 SYCL 的 GPU IVF 能跑出合理的近似搜索结果，但受限于随机访存和 top-k selection，未超过 CPU HNSW。

图 1 的横轴使用对数尺度，可以更清楚地观察不同算法族之间的数量级差异。x86 HNSW 位于最左侧，说明在本数据集上图导航减少候选访问的收益超过了继续优化单候选距离核；x86 IVF、FastScan 和 Flat 依次变慢，反映出“候选裁剪、查表压缩、全量扫描”三类策略的层次关系。ARM 上整体延迟更高，但排序基本一致，说明最终算法结构具有可迁移性；Intel GPU 的 OpenMP target 和 SYCL 位于 CPU IVF 附近，说明 batch scan 的吞吐已经发挥出来，但由于候选组织、距离矩阵回传和 host 端 selection 仍有开销，它们没有超过 CPU HNSW。

表 5 进一步给出各核心算法族的代表性可行点。这里没有把 recall 更高但延迟更大的配置作为最优，因为优化目标始终是 recall@100 $\geq$ 0.95 下的最低 latency。这个选择规则会影响所有后续结论：Flat exact 和若干高 recall 的量化路径虽然结果更接近精确搜索，但它们消耗了过多候选扫描；HNSW 和 IVF 则通过控制访问范围，在跨过目标线后立即停止继续扩大搜索。

表 5: 主要算法在 recall@100 $\geq$ 0.95 下的代表性最优结果

平台	算法	关键参数	latency	recall@100	说明
x86 CPU	HNSW	StdAsync, ef=64, t=32	0.009739 ms	0.954540	全局最快 sweep 结果。
x86 CPU	IVF	n1512, nprobe=32, t=32	0.044471 ms	0.956950	非图索引最快。
x86 CPU	OpenMP CPU IVF	n1512 dynamic, nprobe=32, t=32	0.082060 ms	0.956950	OpenMP host 最优。
x86 CPU	FastScan	block=64, p=750, t=32	0.081048 ms	0.981710	查表压缩路径。
x86 CPU	Flat exact	SSE, t=32	0.130451 ms	1.000000	精确搜索基线。
ARM CPU	HNSW	StdThread, ef=64, t=16	0.085378 ms	0.954280	ARM quick 最优。
ARM CPU	IVF	n1512, nprobe=32, t=32	0.193230 ms	0.956240	ARM 非图索引最快。
ARM CPU	OpenMP CPU IVF	n1512 dynamic, nprobe=32, t=8	0.338880 ms	0.950767	ARM OpenMP host 最优。
ARM CPU	FastScan	block=128, p=500, t=32	0.298396 ms	0.952520	NEON 下最优量化路径。
Intel GPU	OpenMP target IVF	nprobe=81, cap=3840	0.186502 ms	0.950170	严格设备卸载。
Intel GPU	oneAPI/SYCL IVF	nprobe=84	0.210546 ms	0.952790	GPU 编程工具对比。

5 Flat-SIMD 与 Pthread 并行

5.1 算法设计

Flat 搜索是所有 ANN 优化的起点。它对每个 query 枚举所有 base 向量，计算距离并维护 top-100。其优点是结果精确，recall@100=1.0；缺点是计算量和内存访问量最大。Pthread 阶段考虑两种并行方式：query-level 并行和 base-split 并行。

query-level 并行将不同 query 分配给不同线程。每个线程独立维护自己的 top-100 heap，线程间

没有同步，适合 batch query 评价。base-split 并行把同一个 query 的 base 集合切成多个片段，每个线程维护局部 top- p ，最终归并为 top-100。这种方式能降低单 query 延迟，但 p 影响 recall：过小会丢失跨分片的真实近邻，过大会增加归并成本。

Flat 章节中的代码改造集中在工作切分和 top-k 合并。原 SIMD 阶段的距离函数只处理单线程扫描，Pthread 版本在其外层增加 **ThreadRange** 描述每个线程负责的 base 区间，并为每个线程预分配局部候选数组，避免在热循环中动态分配。query-level 路径直接把 query id 分块给线程；base-split 路径则让每个线程输出局部 top- p ，主线程再执行一次小规模归并。为了减少 false sharing，线程私有候选缓冲按线程编号分段存放，不共享 heap 或计数器。

具体到 Pthread 调度，query-level 版本显式创建 T 个 worker，并使用循环式工作分配：第 t 个线程处理 $t, t+T, t+2T, \dots$ 号 query。每个 worker 只写自己负责的 result slot，因此不需要互斥量；主线程 join 后再统一统计 recall。base-split 版本先按线程数计算 base chunk，每个 worker 只扫描自己的连续区间，并把局部 heap 展开为向量保存到 `local\_results[t][qi]`。主线程归并时只处理 $T \times p$ 个候选，而不是重新访问原始 base，因此归并成本可由 p 显式控制。

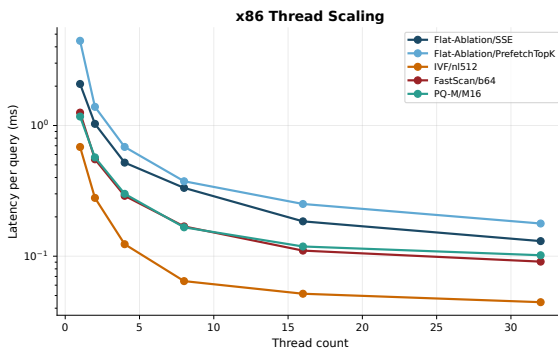
这部分实现保留了精确搜索的可解释性，也暴露出 Pthread 优化的边界。距离计算已经由 SIMD 压缩后，Flat 的主要开销逐渐转向连续内存读取和 top-k 更新；base-split 虽然能并行单个 query，但局部候选越多，最终 merge 越重。因此 Flat 章节不仅是性能基线，也验证了后续 IVF/HNSW 中“先减少候选，再并行处理”的必要性。

Algorithm 1 Flat base-split 并行搜索

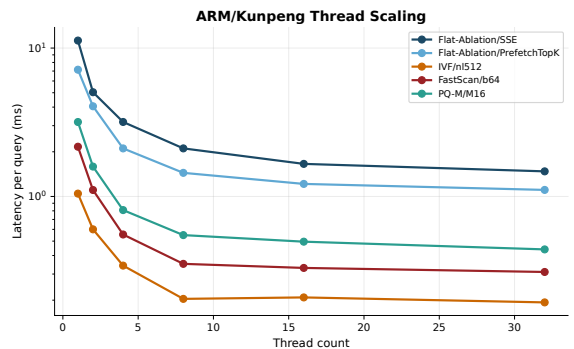
- 1: 输入 query q ，base 集合 X ，线程数 T ，局部候选数 p
 - 2: 将 X 均分为 T 个连续分片
 - 3: **for all** 线程 $t = 1, \dots, T$ **in parallel do**
 - 4: 在本地分片上用 SIMD 距离核扫描候选
 - 5: 维护局部 top- p
 - 6: **end for**
 - 7: 合并所有局部 top- p ，输出全局 top-100
-

5.2 优化过程与实验结果

Flat 路径首先比较 Scalar、AutoVec、SSE、AVX2、NEON-Unroll4 和 PrefetchTopK 等距离核，再在 1、2、4、8、16、32 线程上消融。x86 上 32 线程 SSE 精确搜索达到 0.130451 ms/query；ARM 上最优精确 Flat 路径约 0.717344 ms/query。虽然多线程和 SIMD 都能显著降低 Flat 延迟，但它仍然慢于 IVF 和 HNSW，说明单纯提高吞吐无法替代候选裁剪。



(a) x86 线程扩展



(b) ARM/Kunpeng 线程扩展

图 2: x86 与 ARM/Kunpeng 平台代表性算法的线程扩展曲线。Flat 随线程数下降最明显，量化、IVF 与 HNSW 更早进入访存、候选选择或调度瓶颈。

从线程扩展曲线看，Flat 在 x86 上随线程数增加下降较明显，因为全量扫描的计算量足够大，worker 之间又几乎没有共享写入；但到 16 或 32 线程后，曲线已经不再接近线性下降，说明连续读取 base 向量、维护 top-100 heap 和写回结果开始限制扩展。ARM 上的趋势更保守，主要原因是核心数、内存层次和 NEON SIMD 宽度与 x86 不同，同样的线程数未必对应同样的有效并行度。

Flat 实验给出两个后续优化方向。第一，全量扫描访问所有向量，查询开销由候选规模直接决定，因此仅靠 Pthread 把同样的 N 个候选分给更多线程，只能降低常数，不能改变复杂度。第二，线程数

增加后收益逐渐下降，说明距离计算之外的内存访问、top-k 维护和调度开销开始占主导。因此后续算法重点转向减少候选量，而不是继续堆叠线程。这个判断直接引出了 SQ8/PQ/FastScan 的单候选压缩，以及 IVF/HNSW 的候选裁剪实验。

6 SQ8、PQ、SDC 与 FastScan

6.1 SQ8: 标量量化与 rerank

SQ8 将每个浮点维度量化为 8 bit 整数，查询时先用量化距离快速筛出候选，再用原始浮点向量 rerank。设原始向量维度为 D ，量化后每个维度只需整数运算和查表，单候选扫描成本明显降低。其 trade-off 在于候选数 p ：如果 p 太小，量化误差会使真实近邻被提前丢弃；如果 p 太大，rerank 成本会上升。

实现上，SQ8 路径把量化参数、压缩后的 base code 和原始 base 向量分开保存。第一阶段的近似扫描只读取 8 bit code 和 scale/zero-point，线程私有地维护近似 top- p ；第二阶段 rerank 再回到原始浮点向量，用 SIMD 距离核计算精确距离。这样可以把“近似筛选”和“精确重排”拆成两个可消融阶段，CSV 中分别记录候选数、线程数和最终 latency。Pthread 并行粒度仍以 query-level 为主，因为 SQ8 的 rerank 候选较少，强行在单 query 内继续切分会让 top-k 合并成本占比变高。

实验中 SQ8 在 x86 上最优可行点为 0.296953 ms/query, recall@100=0.989540；ARM 上最优点为 2.308882 ms/query, recall@100=1.0。这说明 SQ8 能保持高 recall，但在本数据集和实现中，候选筛选后的精排仍然偏重，不如 IVF 或 HNSW。

6.2 PQ-ADC 与 PQ-SDC

Product Quantization 将 D 维向量拆分为 M 个子空间，每个子空间用一个 codebook 表示。对 base 向量 x ，PQ 编码为

$$c(x) = (c_1(x), c_2(x), \dots, c_M(x)).$$

ADC (Asymmetric Distance Computation) 在查询时保留 query 的浮点值，构建 query 到每个子空间 codeword 的 LUT，然后对候选编码查表累加：

$$d_{\text{ADC}}(q, x) \approx \sum_{m=1}^M \text{LUT}_m[c_m(x)].$$

SDC (Symmetric Distance Computation) 同时量化 query 和 base，进一步减少运行时浮点计算，但量化误差更大。

Algorithm 2 PQ-ADC 查询

- 1: 输入 query q , PQ codebook, base codes, 候选数 p
 - 2: **for** $m = 1$ to M **do**
 - 3: 计算子查询 q_m 到第 m 个 codebook 的 LUT
 - 4: **end for**
 - 5: **for all** base code $c(x)$ **in parallel do**
 - 6: $s \leftarrow \sum_{m=1}^M \text{LUT}_m[c_m(x)]$
 - 7: 用 s 更新近似 top- p
 - 8: **end for**
 - 9: 对 top- p 候选进行原始向量 rerank, 输出 top-100
-

PQ 的优势是把距离计算从 D 维浮点运算变为 M 次查表；劣势是查表访问模式更随机，线程数增加后可能很快被 cache 和内存带宽限制。因此本阶段重点消融 $M = 8, 12, 16$ 、候选数 p 和线程数，而不是只报告单点结果。

PQ/SDC 的代码改造重点是 LUT 生命周期和线程私有化。ADC 中每个 query 都要构造 M 个子空间 LUT，因此实现上把 LUT 放在线程本地缓冲，避免多线程共享写入；base codes 采用连续数组存储，使一个线程扫描候选时能顺序读取 code。SDC 路径则将 query 也编码后使用预计算表，减少在

线浮点计算，但需要额外记录 query code 和 base code 的对应关系。Pthread 层只并行候选扫描和局部 top- p ，不并行单个 LUT 的构造，因为 LUT 构建占比小，拆得过细会增加调度开销。

LUT 构建本身可以按子空间、query 或 centroid 集合并行，但在本数据集上它不是主导瓶颈。以 $M = 16$ 、每子空间 256 个 codeword、 $D_s = 6$ 为例，一个 query 的 LUT 大约只有 $16 \times 256 \times 6$ 级别的浮点操作；随后的 ADC 扫描则要对大量 base code 执行 $C \times M$ 次查表累加，并维护 top- p 。如果为 LUT 构建再开一层线程，需要在查表扫描前增加 barrier，并且每个线程只得到很小的计算片段。相反，query-level 并行能让每个 worker 完整构造自己的 LUT，再连续扫描候选，减少同步边界，也避免多个线程同时写同一张 LUT。

对查表累加阶段，垂直划分子空间还会引入部分距离归约。若线程 0 处理前 8 个子空间、线程 1 处理后 8 个子空间，它们必须为同一个候选写入临时 partial distance，再归约后才能更新 top-k；这个临时矩阵会放大内存写入，并且 top-k 阈值不能在部分距离阶段可靠更新。本文因此把 PQ 的并行重点放在 batch query 和候选水平划分上，把子空间并行作为复杂性分析而不是最终路径。profiling 中 PQ/FastScan 的 selection 占比也支持这一判断：真正限制性能的是候选数量和 top-k 维护，而不是 LUT 构建的少量 dense compute。

6.3 FastScan 与查表布局

FastScan 继续优化 PQ/SDC 的查表累加。核心思想是将 code 和 LUT 改成更适合 SIMD lane 的布局，使多个候选的子空间查表可以在寄存器中并行累加。实验中 x86 的 FastScan 最优可行点为 block=64, p=750, threads=32，延迟 0.081048 ms/query, recall@100=0.981710；ARM 上最优为 block=128, p=500, threads=32，延迟 0.298396 ms/query, recall@100=0.952520。

FastScan 的实现不再按“一个候选一段 code”的朴素布局扫描，而是按 block 重排多个候选的子空间 code。这样一个 SIMD register 可以同时处理多个候选的同一子空间查表结果。Pthread 层按 block 区间切分工作，每个线程处理完整 block，避免两个线程写同一个 block 的局部分数。该实现使查表路径更接近顺序访问，但也依赖候选数和 block size 的配合：block 太小无法充分利用 SIMD，block 太大则会增加缓存压力和局部 top- p 的维护成本。

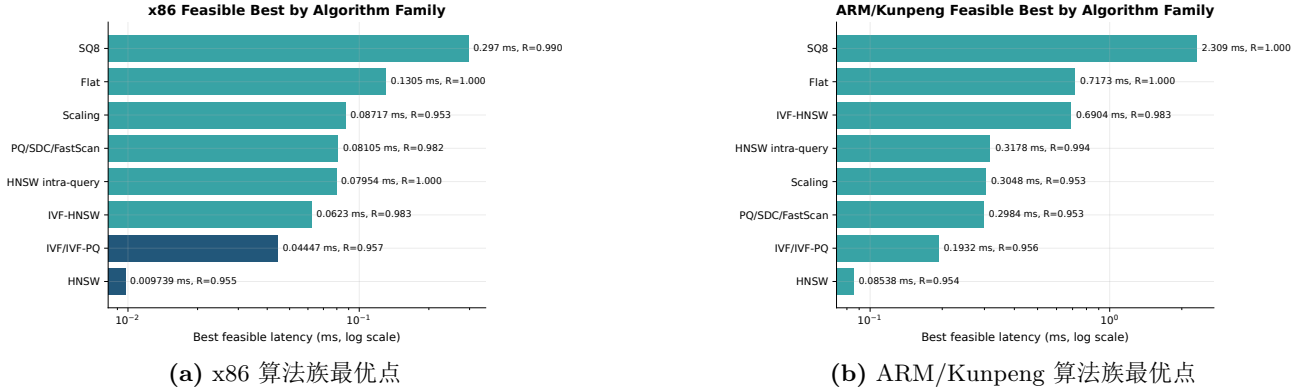


图 3: x86 与 ARM/Kunpeng 平台各算法族在 recall@100 ≥ 0.95 下的最优可行点。HNSW 与 IVF 在双平台上都保持较强的候选裁剪优势，Flat 与部分量化路径主要受候选规模和 selection 成本限制。

从图 3 的两个面板可以看到，FastScan 在两个平台上都明显快于精确 Flat，但没有超过 IVF 和 HNSW。原因是 FastScan 降低了单候选成本，却没有像 IVF/HNSW 那样大幅减少候选数量。它适合作为 SIMD 与 Pthread 结合的高质量消融，但不是最终最优路径。

两张算法族对比图说明，HNSW 的优势需要放在完整算法谱系中理解。Flat 给出精确搜索上界和线程扩展基线；SQ8/PQ/SDC/FastScan 展示压缩距离计算的收益与误差代价；IVF/IVF-PQ 展示通过 coarse quantizer 减少候选集合的效果；HNSW 查询内并行和 IVF-HNSW 则展示图索引内部工作切分的实际代价。最终 HNSW 胜出并不意味着其他实验可省略，因为只有通过这些消融才能说明性能提升来自图结构候选裁剪，而不是来自某个单独 SIMD kernel、线程数或计时口径。

6.4 量化路径的优化复盘

量化路径的优化过程体现了一个典型现象：当距离计算被压缩后，瓶颈会从 arithmetic 转向 memory 和 selection。SQ8 的每维量化非常直接，但为了修正量化误差，需要保留较多候选进行 rerank；PQ-ADC 的 LUT 构建开销较小，主要瓶颈是对 base codes 的随机查表；PQ-SDC 进一步减少查询时

计算，却把误差引入 query 和 base 两侧；FastScan 通过改变数据布局缓解 SIMD 查表成本，但仍然无法减少需要扫描的候选总量。

代码层面，量化类方法的共同约束是线程不能共享临时 LUT、候选堆和 rerank 缓冲。否则即使距离核本身很快，也会在缓存一致性和锁竞争上损失性能。本文将这些状态全部放入 worker-local buffer：SQ8 的近似 top- p 和 rerank 输入数组由线程私有维护；PQ-ADC 的 LUT 按线程分配，query 切换时原地覆盖；FastScan 的 block 分配保证一个 block 只被一个线程处理。这样做牺牲了一定内存占用，但换来了清晰的并发边界和稳定的重复实验结果。

表 6: 量化与查表类方法的代表性结果

平台	方法	参数	latency	recall@100	主要解释
x86	SQ8	p=50, t=32	0.296953 ms	0.989540	recall 高，但精排比例仍偏大。
x86	FastScan	block=64, p=750, t=32	0.081048 ms	0.981710	layout 优化后查表吞吐较好。
x86	FastScan scaling	block=64, p=500, t=32	0.087171 ms	0.952830	更接近目标 recall，延迟略高于最优 IVF。
ARM	SQ8	p=200, t=32	2.308882 ms	1.000000	高 recall 依赖更多候选，延迟不优。
ARM	FastScan	block=128, p=500, t=32	0.298396 ms	0.952520	ARM 上最优量化点，但仍慢于 IVF。

从优化复盘看，PQ/FastScan 的价值不在于最终战胜图索引，而在于展示 SIMD 与 Pthread 的组合边界。它们将单候选计算压到很低之后，继续优化的方向应转向候选生成，而不是继续微调 LUT 累加。因此后续进入 IVF 和 HNSW，是算法结构层面的必要升级。

7 IVF 与 IVF-PQ

7.1 倒排索引结构

IVF (Inverted File) 先用聚类把 base 集合划分为 $nlist$ 个簇，并保存每个簇的质心和倒排链。查询时先计算 query 到所有质心的距离，选择最近的 $nprobe$ 个簇，只扫描这些簇中的向量。其核心复杂度由全量 N 降到约 $N \cdot nprobe / nlist$ ，代价是可能漏掉未被 probe 到的真实近邻。

IVF 的代码改造围绕倒排链数据结构展开。CPU 版本将每个 list 存为连续候选 id 数组，并额外保存 list offset，查询时先得到 $nprobe$ 个 list id，再把这些 list 的候选区间交给线程扫描。每个线程维护自己的局部 top-100，扫描后归并。这样的设计比在倒排链上加锁更直接，因为查询阶段 list 只读，写入只发生在线程私有候选堆中。IVF-PQ 复用同一套 list 结构，只是候选距离由原始 SIMD 距离替换为 PQ 查表距离。

为了让 CPU 与 GPU 路径共享实验口径，IVF 的结果记录同时保存 $nlist$ 、 $nprobe$ 、线程数和候选上界。x86/ARM 的 Pthread IVF 扫描完整倒排链；Intel GPU 的 OpenMP target IVF 在同一索引思想上增加 candidate cap，用来控制设备端每个 query 的最大工作量。这使得 IVF 章节既承担非图索引实验，也为后续 OpenMP target 加速器路径提供数据结构基础。

Pthread IVF 的 query-level worker 与 Flat 一致采用循环式 query 分配，每个线程调用同一个 `ivf\_search` 逻辑，内部先计算 query 到质心的距离，再扫描 $nprobe$ 个倒排链。为了探索单 query 内并行，代码还实现了 cluster-parallel fine scan：先选出 `probe\_ids`，再让多个线程按 `pi = tid, tid+T, ...` 分担倒排链扫描，每个线程维护本地 heap。最终结果仍要归并局部 heap，因此它主要用于消融单 query 并行是否值得，而最终最优仍来自 query-level 并行。

Algorithm 3 IVF 查询

- 1: 输入 query q , 质心集合 C , 倒排链 $\mathcal{L}$, 参数 $nprobe$
- 2: 计算 q 到所有质心的距离
- 3: 选择最近的 $nprobe$ 个质心编号 P
- 4: **for all** $j \in P$ **do**
- 5: 扫描倒排链 $\mathcal{L}_j$ 中的候选
- 6: 使用 SIMD 距离核或 PQ 近似距离更新 top-100
- 7: **end for**
- 8: 输出 top-100

7.2 $nprobe$ 的 trade-off

$nprobe$ 是 IVF 最关键的旋钮。增大 $nprobe$ 会提高 recall, 但扫描候选数也近似线性增加。本文在 CPU 上消融 $nlist=64, 128, 256, 512$ 与 $nprobe=1, 2, 4, 8, 16, 32$, 在 GPU 上围绕 $nlist=2048$ 细化 $nprobe=79..84$ 和 candidate cap。

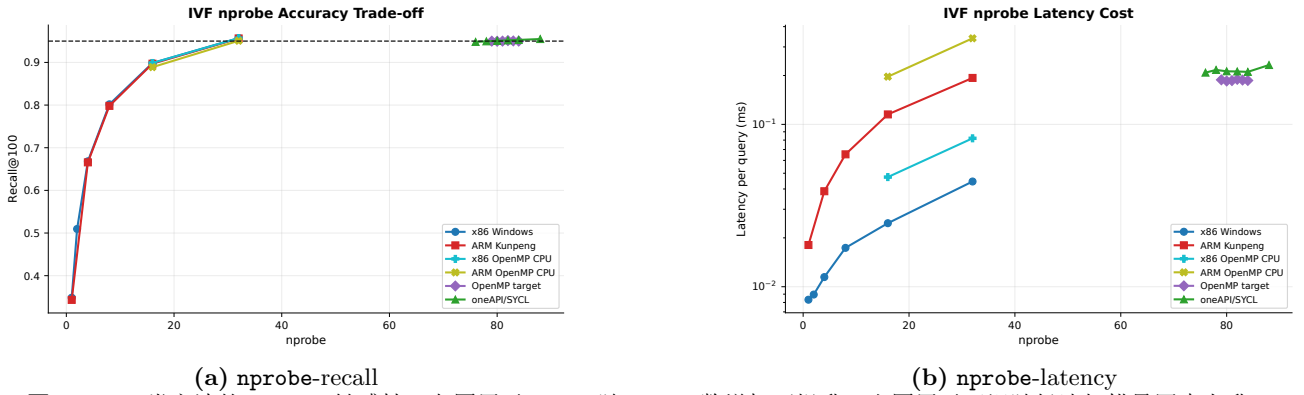


图 4: IVF 类方法的 $nprobe$ 敏感性。左图展示 recall 随 probe 数增加而提升, 右图展示延迟随候选扫描量同步上升。

x86 CPU 上, 最优 IVF 为 $nlist=1512$, $nprobe=32$, $threads=32$, 延迟 0.044471 ms/query, $recall@100=0.956950$ 。ARM/Kunpeng 上最优 IVF 同样为 $nlist=1512$, $nprobe=32$, $threads=32$, 延迟 0.193230 ms/query, $recall@100=0.956240$ 。这说明 IVF 的最佳 $nprobe$ 在两个平台间具有一定可迁移性, 但绝对延迟受 SIMD 宽度、内存层次和线程运行时影响明显。

IVF 的实现重点在于让 $nprobe$ 的算法含义和线程执行边界一致。coarse 阶段只负责选择倒排链, 不直接参与最终 top-100; fine scan 阶段才把候选 id 映射到原始向量或 PQ code, 并在线程私有 heap 中维护局部结果。这样记录 CSV 时可以分别解释 recall 上升来自 probe 链增多, latency 上升来自 fine scan 候选增加。对于 GPU IVF, candidate cap 又进一步把这个候选量限制成固定矩阵形状, 使 OpenMP target kernel 可以规则并行; 代价是 cap 过小会截断候选、降低 recall, cap 过大又会增加设备扫描和回传距离矩阵的成本。

7.3 IVF-PQ 与局部 PQ

IVF-PQ 将“候选裁剪”和“量化距离”结合起来。全局 IVF-PQ 先对全体 base 训练 PQ, 再在倒排候选内查表; 局部 IVF-PQ 则在每个簇内训练或使用更贴合局部分布的 PQ。理论上, 局部 PQ 能降低量化误差; 实践中, 它会增加构建成本, 并且每个簇的训练样本更少, 稳定性不如全局 PQ。

两种 IVF-PQ 结构的取舍可以从误差、缓存和构建代价三方面理解。全局 PQ 的优点是 codebook 统一, 所有倒排链共享同一套 LUT, 查询时只需为一个 query 构建一次 LUT, 然后在不同 list 中复用; 缺点是全局 codebook 未必贴合每个簇内部的残差分布。局部 PQ 的优点是每个 list 内部的量化误差可能更低, 尤其当簇内分布比较紧时更有意义; 缺点是每个 list 的 codebook 和 LUT 都不同, 查询多个 list 时要频繁切换表, 构建阶段也要为每个 list 训练或保存额外码本。对 DEEP100K 这种 100K 规模, 局部 PQ 的构建和缓存复杂度没有换来足够低的查询延迟, 因此最终非图最优仍选择 IVF 原始距离精排。

实验结果表明, IVF-PQ 系列是理解索引与量化叠加的重要消融, 但在 DEEP100K 上最终最快的非图索引仍是 IVF + 原始距离精排。这说明当 base 规模为 100K、维度为 96 时, 减少扫描簇数比进

一步压缩单候选距离更重要。

8 HNSW 图索引

8.1 HNSW 查询过程

HNSW (Hierarchical Navigable Small World graph) 用多层近邻图组织 base 向量。查询从最高层入口点开始贪心下降, 到 Layer 0 后维护候选队列和结果队列进行搜索。参数 `ef` 控制候选队列大小: 较大的 `ef` 提高 recall, 但会访问更多节点并增加 latency。

Algorithm 4 HNSW 标准查询

```
1: 从顶层入口点 ep 开始
2: for level 从最高层降到 1 do
3:   在当前层贪心移动到离 query 更近的邻居
4: end for
5: 在 Layer 0 维护候选队列 C 和结果队列 R
6: while C 中仍有可能改进 R 的节点 do
7:   弹出当前最近候选, 访问其邻居
8:   若邻居更优则加入 C 和 R, 并保持  $|R| \leq ef$ 
9: end while
10: 返回 R 中 top-100
```

8.2 query-level 并行

HNSW 查询内部存在强依赖: 下一步访问哪个节点取决于当前候选队列状态。因此最稳定的多线程方式是 query-level parallel, 即不同线程处理不同 query, 共享只读图索引。x86 上 StdAsync, `ef=64, threads=32` 的 sweep 延迟为 0.009739 ms/query, `recall@100=0.954540`; ARM 上 StdThread, `ef=64, threads=16` 的 quick 延迟为 0.085378 ms/query, `recall@100=0.954280`。

实现上, HNSW 查询线程之间只共享图结构和向量数据, 候选队列、结果队列、visited 标记都在线程私有内存中维护。x86 版本比较了 `std::async`、`std::thread` 和固定线程分块; ARM/Kunpeng 版本保留同样的逻辑, 但默认 quick sweep 用较少 query 选择线程数, 再用 final 路径验证。这个改造避免了多个线程竞争同一个候选队列, 也让 HNSW 的 Pthread 优化集中在 query 分发、线程复用和私有缓冲复用上。

HNSW 的 visited 标记如果每次 query 都重新清空完整数组, 会引入额外内存写入。因此实现中使用线程私有的访问版本号或局部访问列表, 只清理实际访问过的节点。结果队列使用固定容量结构保存 `ef` 个候选, 避免在搜索热路径中频繁申请内存。上述工程改造对延迟影响很大, 因为最终 HNSW 单 query 访问节点数已经很少, 任何线程调度和内存初始化开销都会被放大。

在标准 HNSW query-level 版本中, `std::thread` 路径显式创建 *T* 个 worker, 并按 query id 间隔分配工作; `std::async` 路径把同样的分配方式交给 C++ 运行时调度, 并用 future 等待所有 worker 结束。每个 query 调用 `searchKnn` 后, 将 `hnswlib` 返回的 heap 转换为统一的 ANN top-k 结构, 再计算 `recall@100`。这保证 HNSW、IVF 和 Flat 的结果评价函数一致, 避免因为不同库返回顺序不同而影响 recall 统计。

表 7: C++ 标准库线程工具在 HNSW query-level 上的对比

平台	工具	参数	latency	recall@100	分析
x86	<code>std::thread</code>	<code>ef=64,t=32</code>	0.011492 ms	0.954540	显式 worker 稳定, 但略慢于 <code>async</code> 。
x86	<code>std::async</code>	<code>ef=64,t=32</code>	0.009739 ms	0.954540	本地运行时对短 query 分发更轻。
ARM	<code>std::thread</code>	<code>ef=64,t=16</code>	0.085378 ms	0.954280	16 线程最稳, 继续增线程收益下降。
ARM	<code>std::async</code>	<code>ef=64,t=8</code>	0.090980 ms	0.954280	8 线程较优, 但高线程数调度波动更明显。

表 7 展示了 C++ 标准线程工具的跨平台差异。二者使用同一个 HNSW 查询函数和同一个 recall 统计过程, 差异来自任务提交、worker 生命周期和 future/join 等等待机制。x86 上 `std::async` 在 32

线程下略快，说明本地标准库运行时能较好地复用或调度这些短任务；ARM 上 `std::thread` 在 16 线程更稳，说明远端平台上显式 worker 与固定 query 分配更容易控制调度成本。这一结果也解释了为什么线程工具本身需要作为实验维度，而不能把 C++ 标准库多线程视为与 Pthread/OpenMP 完全等价的黑盒。

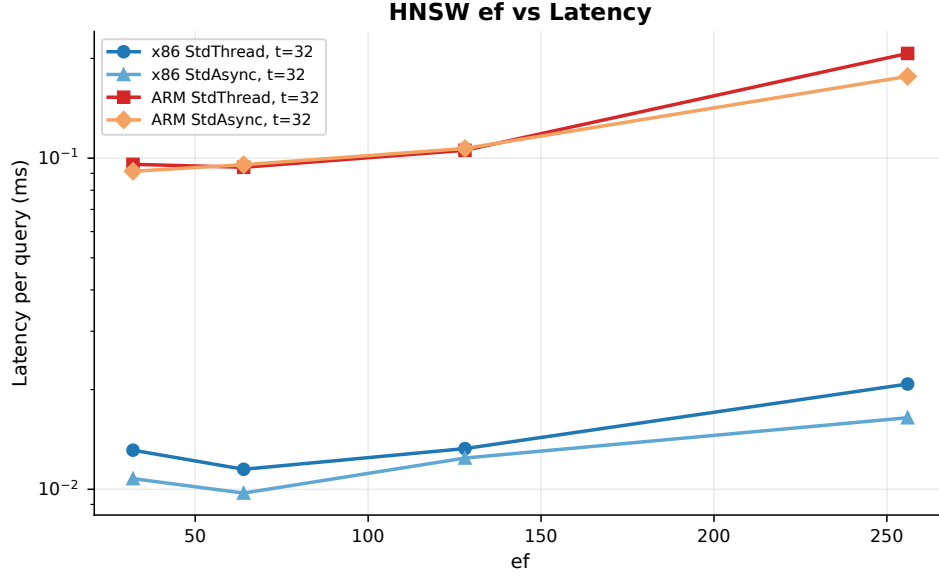


图 5: HNSW 中 `ef` 对延迟的影响。跨过目标线后继续增大 `ef` 会迅速增加搜索成本。

从图 5 可以看出，`ef=64` 是两个平台上兼顾 recall 和 latency 的稳定点。更大的 `ef` 提高 recall，但已经偏离 $\text{recall}@100 \geq 0.95$ 下最小 latency 的目标。

8.3 查询内并行探索

图索引查询内并行包括边划分、Layer 0 点划分、多入口点并行，以及 IVF+HNSW 的嵌套结构。本文分别实现并测试了这些策略。其共同思路是把一个 query 的 Layer 0 搜索拆给多个线程，再合并局部结果。

边划分版本把当前扩展节点的邻接边分给多个线程计算距离，然后合并可进入候选队列的邻居；点划分版本把 Layer 0 候选点集合划成多个子集，各线程在子集内维护局部队列；多入口点版本从多个入口同时启动搜索，最后归并局部 top-100；IVF+HNSW 则先用 IVF 粗分簇，再在每个子簇内部运行一个较小的 HNSW。代码上这些变体都使用线程私有队列和 visited 标记，避免共享队列加锁成为唯一瓶颈。它们的差别主要体现在子搜索构造方式和最终候选归并方式。

Algorithm 5 Layer 0 查询内并行的一般形式

- 1: 输入 query q , HNSW Layer 0 图, 线程数 T
 - 2: 按入口点、边集合或点集合构造 T 个子搜索
 - 3: **for all** 子搜索 t **in parallel do**
 - 4: 在子图或子候选集合上执行局部搜索
 - 5: 输出局部 top-100 或局部候选队列
 - 6: **end for**
 - 7: 合并所有局部结果，并计算最终 top-100
-

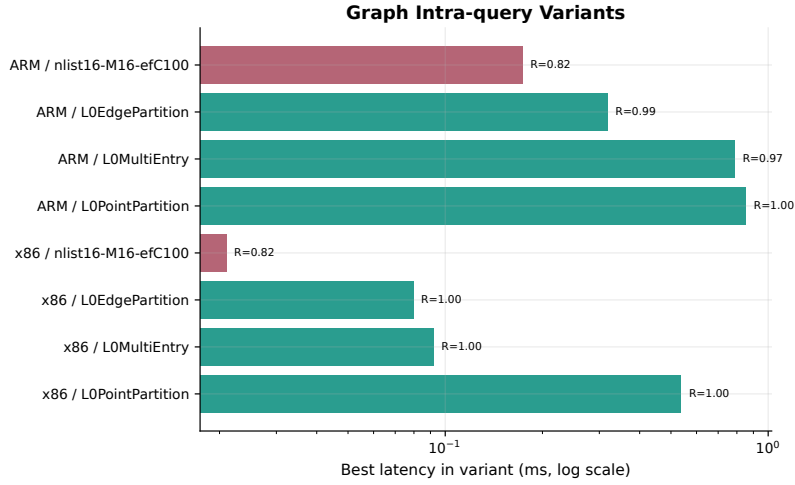


图 6: HNSW 查询内并行和 IVF-HNSW 变体的最快点。绿色表示可行点，紫红色表示低于目标线。

图 6 显示，查询内并行并没有超过标准 query-level HNSW。部分变体能获得高 recall，但延迟明显更高；少数低延迟点 recall 只有约 0.82，未进入最终候选集合。这一现象符合 HNSW 的算法结构：图搜索的价值来自自适应剪枝，简单切分点或边会削弱队列驱动的导航过程，并引入局部队列合并开销。

表 8: HNSW 查询内并行变体的最快记录

平台	变体	参数	latency	recall@100	解释
x86	Layer0EdgePartition	ef=128, t=4	0.079540 ms	0.999820	recall 高，但慢于标准 HNSW。
x86	Layer0MultiEntry	ef=128, entries=2	0.091911 ms	1.000000	多入口增加覆盖，也增加重复搜索。
x86	Layer0PointPartition	partitions=8	0.536787 ms	1.000000	近似退化为分区精扫，延迟高。
x86	IVF-HNSW	nprobe=8, t=32	0.062297 ms	0.982780	分片并行可行，但慢于全局 HNSW。
ARM	Layer0EdgePartition	ef=128, t=8	0.317768 ms	0.994480	recall 较高，但同步与合并较重。
ARM	IVF-HNSW	nprobe=8, t=32	0.690392 ms	0.983180	增加 probe 后召回恢复，但延迟大幅上升。

这些结果说明图索引查询内并行的难点。x86 上可行的 IVF-HNSW 点为 $nprobe=8, t=32$ ，延迟 0.062297 ms，已经比低 $nprobe$ 时可靠得多，但仍约为标准 HNSW query-level 的 6.4 倍；ARM 上同一思路恢复 recall 后延迟达到 0.690392 ms，也明显慢于全局 HNSW。边划分和多入口点能提高覆盖，但它们把“一个全局候选队列”变成多个局部队列，最终仍要去重和归并。若希望真正提升 HNSW 单 query 并行性能，简单划分 Layer 0 还不够，需要重新设计 visited 标记、候选队列共享方式和终止条件；否则线程间并行会增加大量重复访问，反而削弱 HNSW 的导航优势。

9 OpenMP CPU 并行化

9.1 OpenMP host CPU 的实现范围

本文将 OpenMP 分成两条独立路线：本节讨论 host CPU 上的普通 OpenMP 并行，后文第 10 节讨论 OpenMP target 加速器卸载。二者的目标不同：OpenMP CPU 用于和 Pthread 比较共享内存多线程表达方式、schedule 策略和线程管理开销；OpenMP target 则用于分析计算移动到 GPU 等加速器设备后的端到端收益。

OpenMP CPU 版本没有重新发明 ANN 算法，而是在与 Pthread 相同的候选生成和 recall 统计口径下替换线程组织方式。Flat query-level 路径在 query 循环外使用 `##pragma omp parallel for schedule(static/dynamic)`，每个 iteration 独立处理一个 query；Flat base-split 路径在单个 query 内把 base 区间分给多个 OpenMP worker，每个线程维护局部 top- p ，最后归并为 top-100；PQ-ADC 路径把每个 query 的 LUT 放在线程私有缓冲中，避免多个 worker 共享查表数组；IVF 路径先在每个 query 内选择 $nprobe$ 个倒排链，再由 OpenMP 并行处理不同 query 的 coarse+fine scan。

为了让 OpenMP 结果能与 Pthread 公平比较，host CPU 版本也复用同一套 SIMD 距离核和 top-k 评价函数。线程并行层只处理工作划分，不改变候选集合的语义。这样做有两个好处：第一，

OpenMP 与 Pthread 的 recall 差异只来自参数和候选裁剪，不来自不同评价代码；第二，static 与 dynamic schedule 的差异可以被解释为调度策略差异，而不是算法差异。

9.2 schedule、划分方式与一致性

OpenMP CPU 的消融重点是 schedule 与工作粒度。对 Flat query-level，所有 query 的候选数相同，static schedule 理论上足够；但实际运行中，top-k 更新、cache 状态和系统调度会产生轻微差异，因此本文同时测试 dynamic。对 IVF，倒排链长度不均会导致不同 query 的 fine scan 工作量不同，dynamic schedule 更容易把长 query 分散给空闲线程。对 base-split，OpenMP worker 的局部结果需要合并，一致性由三条规则保证：base 与 query 只读，线程只写自己的局部 heap，最终 top-100 只在并行区结束后由主线程归并。

表 9: OpenMP schedule 在 IVF nprobe=32 上的影响

平台	schedule	1T latency	最优线程	最优 latency	recall@100	分析
x86	static	0.646106 ms	16T	0.098325 ms	0.956950	固定切分成本低，但长 query 会形成等待。
x86	dynamic	0.569579 ms	32T	0.082060 ms	0.956950	动态分发缓解倒排链不均，成为 OpenMP CPU 最优。
ARM	static	1.536323 ms	8T	0.416483 ms	0.950767	继续增加线程未在 quick sweep 中带来收益。
ARM	dynamic	1.561803 ms	8T	0.338880 ms	0.950767	负载均衡收益高于 runtime 成本。

表 9 给出了 schedule 选择的直接证据。x86 上 dynamic 在 32 线程时比 static 快约 22%，ARM 上 dynamic 在 8 线程时比 static 快约 19%。这说明 IVF 的 query 循环虽然表面上每次迭代都是“处理一个 query”，但每个 query 选中的倒排链长度并不相同，固定分配会造成尾部等待。dynamic 的代价是 runtime 需要维护任务队列和隐式 barrier；当候选扫描足够重时，这个代价可以被负载均衡收益覆盖。对于 guided schedule，它更适合高斯消去那类迭代工作量随循环编号呈规律变化的场景；IVF 的工作量由 query 与聚类分布共同决定，不随 query id 单调变化，因此本文重点比较 static 与 dynamic 这两个更直接的策略。

OpenMP 的归约和同步也需要按 ANN 语义重新理解。普通数值归约可以用 reduction(+)，但 ANN 的 top-k 是有序集合归并，既要比较距离，也要保留 id 并去重，不能用简单标量归约替代。本文因此避免在热循环中使用 critical 或共享 heap，而是让每个 worker 维护局部 top-k，退出并行区后再合并。隐式 barrier 只保留在必须完成所有局部结果之后的位置；在单个 query 内反复 barrier 会把同步成本放大到与搜索本身同一数量级。

Algorithm 6 OpenMP CPU IVF query-level 并行

- 1: 输入 batch queries、IVF 索引、线程数 T 、schedule 策略
- 2: 设置 `omp_set_num_threads(T)`
- 3: **for all** query id i **with OpenMP parallel for do**
- 4: 计算 query 到所有质心的距离，选择 `nprobe` 个倒排链
- 5: 在线程私有 heap 中扫描倒排链候选并维护 top-100
- 6: 将该 query 的 top-100 写入独立结果槽
- 7: **end for**
- 8: 汇总 recall@100 与平均 latency

这种写法与 Pthread 固定 worker 的主要区别在于工作分配由 OpenMP runtime 管理。它减少了手写线程代码，但也引入调度器、隐式 barrier 和循环 chunk 管理成本。对候选扫描很重的配置，这部分成本可以被摊薄；对 HNSW 这类单 query 已经很快的路径，OpenMP 循环调度未必优于显式线程分块。因此本文没有把 OpenMP CPU 改写成 HNSW 最终路径，而是把它用于并行模型对比和 schedule 分析。

9.3 OpenMP CPU 实验结果

x86 与 ARM 的 OpenMP CPU 最优点均来自 IVF dynamic schedule。x86 上 OpenMP-CPU-IVF, nl512-dynamic, nprobe=32, threads=32 达到 0.082060 ms/query, recall@100=0.956950;

ARM/Kunpeng 上 `threads=8` 达到 0.338880 ms/query, `recall@100=0.950767`。这两个点的 recall 均高于 0.95, 也都与 Pthread IVF 使用相同的 `nlist=512`, `nprobe=32`, 说明算法参数可迁移; 差异主要来自线程运行时、SIMD 宽度和内存层次。

表 10: Pthread 与 OpenMP CPU 的同平台对比

平台	实现	参数	latency	recall@100	解释
x86	Pthread IVF	<code>n1512, nprobe=32, t=32</code>	0.044471 ms	0.956950	手写 worker 分配更轻, 非图 CPU 最快。
x86	OpenMP CPU IVF	<code>dynamic, nprobe=32, t=32</code>	0.082060 ms	0.956950	调度表达简洁, 但运行时开销较高。
ARM	Pthread IVF	<code>n1512, nprobe=32, t=32</code>	0.193230 ms	0.956240	ARM 非图索引最优。
ARM	OpenMP CPU IVF	<code>dynamic, nprobe=32, t=8</code>	0.338880 ms	0.950767	quick sweep 中跨过目标线的最低延迟。

OpenMP CPU 的结果说明, 普通 OpenMP 可以在相同 ANN 算法上复现实用的 recall-latency trade-off; 但它并没有超过手写 Pthread IVF。原因主要有三点。第一, Pthread 版本在 worker 内复用线程私有缓冲, 工作分配更接近固定分块; OpenMP dynamic schedule 为负载均衡付出了额外调度成本。第二, IVF 的 coarse selection 与 top-k 维护并不是纯规则循环, OpenMP 只能并行外层 query, 无法消除每个 query 内的 heap 更新。第三, ARM quick 环境线程数较少, OpenMP runtime 的相对开销更容易被放大。

因此, OpenMP CPU 在本文中主要作为共享内存线程模型的对照: 它提供了较简洁的循环并行表达, 也展示了 runtime schedule 与隐式同步的代价; 而最终 CPU 查询路径仍选择 HNSW Pthread/C++ 标准库 query-level 并行, 因为该路径在 `recall@100 ≥ 0.95` 下 latency 更低。

10 OpenMP Target 与 oneAPI/SYCL

10.1 加速器实现目标

OpenMP target 路径与 OpenMP CPU 路径分开讨论。这里关注的是计算从 host 端移动到 Intel GPU 后, 规则 batch scan 能否降低 IVF 查询延迟。GPU 路径采用 batch IVF: host 端负责质心训练、倒排链构建和 candidate 填充, device 端负责批量 query 的候选距离扫描, host 端回收距离矩阵后执行最终 top-k 选择。这样可以把 GPU 的规则并行能力集中用在最重的候选扫描上, 同时避免把 HNSW 这类动态优先队列搜索直接搬到设备端。

代码实现中, 质心矩阵、倒排链 offset、候选 id 数组和 base 向量被整理为连续内存, 并在 target data 区域映射到设备。每个 batch 先在 host 侧根据 `nprobe` 和 candidate cap 填充候选 id; 随后 target kernel 展开为 `q\_count * candidate\_cap` 个并行迭代, 每个迭代计算一个 query-candidate 距离。无效候选槽写入极大距离, 保证输出距离矩阵形状固定, 便于 host 端统一执行 top-k selection 和 recall 统计。

10.2 OpenMP target IVF

OpenMP target 实验严格卸载到 Intel GPU 加速器设备, 运行时设置 mandatory offload, 并在 target 区域调用 `omp\_is\_initial\_device()` 检查当前执行设备不是 host。算法上采用 `nlist=2048` 的 IVF, 围绕 `nprobe` 和 candidate cap 进行精细搜索。与 CPU Pthread 不同, OpenMP target 的主要优化不是增加 host 线程, 而是控制设备端扫描候选数、提高 batch 内并行度, 并把 scan、selection 和回传三段时间分开统计。

这部分代码刻意保持 kernel 简单: 设备端不使用 STL 容器、不动态申请内存、不维护复杂优先队列, 而是将二维工作空间展平为 `q\_count * candidate\_cap` 个迭代。每个迭代只做三件事: 读取候选 id, 判断 slot 是否有效, 计算一个 query 与一个 base 向量的距离并写入距离矩阵。这样的写法更接近 GPU 的规则数据并行模型, 也更容易在 Intel OpenMP target 编译链下稳定使用普通 `-O2`。最终 top-100 selection 留在 host 端执行, 用于复用 CPU IVF 的 recall 统计函数; selection 也作为后续可优化瓶颈单独讨论。

Algorithm 7 OpenMP target batch IVF

- 1: Host 构建 IVF 质心、倒排链和紧凑候选数组
 - 2: Host 按 `nprobe` 和 `candidate cap` 填充 batch 候选矩阵
 - 3: 将 batch queries、candidate ids 和 base vectors 映射到 target device
 - 4: **for all** device 端的 (*query, candidate slot*) 并行迭代 **do**
 - 5: 读取候选 id; 无效 slot 写入极大距离
 - 6: 对有效候选执行 SIMD-friendly 距离累加, 写出距离矩阵
 - 7: **end for**
 - 8: 将距离矩阵回传 host
 - 9: Host 执行 final top-100 selection 与 recall 统计
-

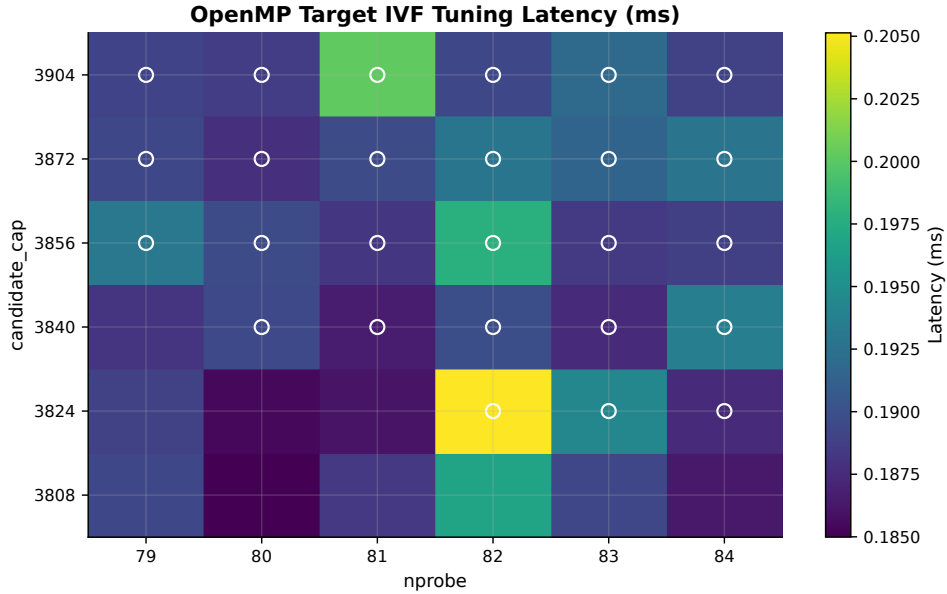


图 7: OpenMP target IVF 的 `nprobe` 与 `candidate cap` 调参热力图, 记录 latency 与可行点分布。

最优 OpenMP target 配置为 `nprobe=81`, `candidate cap=3840`, 延迟 0.186502 ms/query, `recall@100=0.950170`。这个点非常接近目标线, 说明优化方向已经从“让 GPU 跑起来”转为“减少不必要候选扫描”。

从实现角度看, OpenMP target 路径的关键修改有三点。第一, 倒排链不使用 CPU 侧的指针结构, 而是改为 `offsets + ids + vectors` 的扁平数组, 便于设备端按 `offset` 定位候选区间。第二, kernel 内部避免动态分配和复杂 STL 容器, 每个并行迭代只做一次连续向量点积并写回距离, 保证设备端代码可被普通 `-O2` 编译。第三, `candidate cap` 既是算法参数, 也是设备端工作量和回传距离矩阵的上界, 可以防止少数超长倒排链拖慢整个 batch。

这一实现把 OpenMP target 的收益和代价都显式暴露出来。设备端扫描时间对应 `target teams distribute parallel for` 的实际工作, `candidate` 填充时间反映 host 侧根据倒排链构造固定形状候选矩阵的成本, `final top-k/rerank` 时间反映回到 host 后根据精确候选距离维护 top-100 的成本。由于设备端已经对候选原始向量计算精确距离, 这一路径没有额外的二次原向量 rerank kernel; 报告中把 post-scan top-k 作为 rerank-equivalent 阶段汇报。图 8 中可以看到, GPU 路径并不是单一 kernel 时间, 而是由 candidate 组织、设备扫描、距离回传和最终重排共同决定。后续优化空间也很明确: 若要进一步提升 OpenMP target, 需要把 block-level top-k 或 partial selection 下沉到设备端, 减少回传距离矩阵规模。

10.3 与 oneAPI/SYCL 的比较

oneAPI/SYCL 路径采用相近的 Intel GPU batch IVF 思路。补齐 host coarse selection 与候选填充计时后, 正式 O2 结果中 SYCL 最优为 `nprobe=84`, 延迟 0.210546 ms/query, `recall@100=0.952790`。它略慢于 OpenMP target 最优点, 但两者的瓶颈一致: 候选 scan 是主要耗时; SYCL 的 host coarse selection 约为 27.471 μ s/query, candidate fill 与 query layout 约为 0.075 μ s/query, final top-k/rerank

为 $4.622 \mu\text{s}/\text{query}$ ，说明该路径的额外主机侧开销主要来自质心选择和设备扫描，而不是候选矩阵填充。

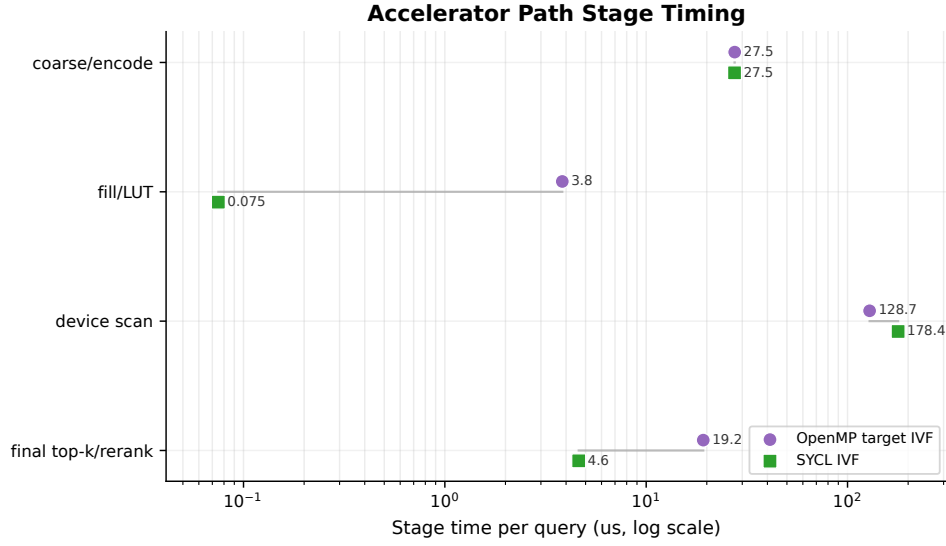


图 8: OpenMP target 与 oneAPI/SYCL 的阶段耗时分解。GPU IVF 已在设备端计算候选精确距离，final top-k/rerank 表示扫描后的最终重排阶段。

GPU 结果说明，batch IVF 的设备端距离扫描已经具备可观吞吐，但端到端延迟仍高于 x86 CPU HNSW。原因有四点。第一，HNSW 在算法层面访问的节点数量远少于 IVF candidate cap 下的候选矩阵，GPU 扫描再快也仍要处理更多候选。第二，GPU kernel 只覆盖候选距离计算，coarse quantizer、candidate 填充、距离矩阵回传和最终 top-100 重排仍在 host 侧参与端到端时间。第三，IVF 倒排链长度不均，candidate cap 为了保持规则矩阵形状会引入无效槽和截断权衡，增加了对参数的敏感性。第四，Intel 集显与 CPU 共享内存层次和功耗预算，适合规则 batch scan，但不具备足以抵消 HNSW 图剪枝优势的候选吞吐差距。因此，GPU 路径的主要瓶颈不是单次距离计算，而是候选规模、host-device 协作和 top-k 位置仍偏 host 端。

表 11: Intel GPU 上 IVF 加速器路径对比

后端	参数	latency	recall@100	build(s)	分析
OpenMP target	nprobe=81, cap=3840	0.186502 ms	0.950170	10.386833	候选扫描、回传和 host selection 共同决定端到端时间。
oneAPI/SYCL	nprobe=84, cap=9929	0.210546 ms	0.952790	17.303400	编程模型更灵活，但本实验中略慢于 OpenMP target。
OpenCL C	nprobe=88	0.260130 ms	0.954270	279.108000	作为底层 GPU API 参照，构建和调参成本更高。
CUDA Flat	exact scan	0.410661 ms	1.000000	0.000000	精确扫描基线，未作为最终加速器路径。

这一对比说明 OpenMP 的两个层次需要分开解释：OpenMP CPU host 描述共享内存线程调度，OpenMP target 描述设备端 batch kernel 和 host-device 协作。CPU 上最优是 HNSW，host OpenMP 的代表结果是 IVF dynamic schedule，GPU 上最适合展示的是 batch IVF。

11 x86 与 ARM/Kunpeng 跨平台分析

11.1 算法最优点是否迁移

跨平台对比中，IVF 的最优参数具有较强一致性：x86 和 ARM 都选择 nl512，nprobe=32。这说明在 DEEP100K 上，倒排簇数量与 recall 目标之间的关系主要由数据分布决定，而不是平台决定。不同平台影响的是绝对 latency：ARM IVF 为 0.193230 ms/query，约为 x86 IVF 的 4.35 倍。

OpenMP CPU 也呈现同样的参数一致性：两个平台最优都为 OpenMP-CPU-IVF，nl512-dynamic，nprobe=32。x86 OpenMP CPU 延迟为 0.082060 ms/query，ARM OpenMP CPU 延迟为 0.338880

ms/query, ARM 约为 x86 的 4.13 倍。这个比例与 Pthread IVF 的跨平台比例接近, 说明瓶颈主要来自相同的 IVF 候选扫描和内存层次差异, 而不是某个单独线程接口的偶然结果。

HNSW 的最优算法族也一致, 但最佳线程工具和线程数发生变化。x86 上 `std::async` 与 32 线程更快; ARM 上 `std::thread` 与 16 线程更稳。这说明当单 query 代价已经很低时, 线程运行时开销、核心数和调度策略会影响最终选择。

11.2 跨平台性能差异的来源

跨平台差异需要把算法访问模式和硬件执行模型分开看。x86 平台的主要优势来自更高单核频率、更宽的 SIMD 执行能力、更大的乱序执行窗口, 以及更积极的 C++/OpenMP 运行时调度; ARM/Kunpeng 平台的优势是核心数稳定、服务器运行环境可控, 但 NEON 向量宽度、单核峰值和内存层次与本地 x86 不同。不同算法对这些因素的敏感性并不相同, 因此不能用一个固定倍率解释所有结果。

表 12: x86 与 ARM/Kunpeng 代表路径的延迟倍率

路径	x86 latency	ARM latency	ARM/x86	主要原因
Flat exact	0.130451 ms	0.717344 ms	5.50×	全量连续扫描受 SIMD 宽度、内存带宽和 top-k 更新共同影响。
FastScan	0.081048 ms	0.298396 ms	3.68×	查表布局降低计算量后, ARM 的相对差距小于 Flat, 但 scan 与 selection 仍受缓存限制。
Pthread IVF	0.044471 ms	0.193230 ms	4.35×	nlist/nprobe 相同, 候选规模一致, 差异主要来自 fine scan、SIMD 与内存层次。
OpenMP CPU IVF	0.082060 ms	0.338880 ms	4.13×	算法参数一致, dynamic schedule 的调度成本在两平台都存在。
HNSW final	0.009316 ms	0.070879 ms	7.61×	随机图访问和低单 query 成本放大了 cache miss、分支和线程运行时差异。

表 12 中 HNSW 的跨平台倍率最大, 并不说明 HNSW 在 ARM 上失效, 而是说明它把候选数量压得很低后, 剩余开销更依赖随机访存、优先队列维护和线程运行时。Flat 与 IVF 的访问更规则, ARM/x86 倍率主要反映 SIMD 与内存带宽; HNSW 的访问路径由 query 决定, 邻接点访问不连续, cache 预取效果较弱, 因此平台缓存层次和分支预测差异会被放大。FastScan 的倍率较低, 是因为它通过 block layout 改善了查表局部性, 减少了浮点距离核差异, 但它仍没有像 HNSW 那样减少候选数量。

问题规模在 ANN 中表现为三类控制量: query batch 规模、候选访问规模和线程规模。x86 sweep 使用 1000 个 query, 能较好摊薄线程启动和计时噪声; ARM quick sweep 使用 500 个 query 筛选参数, 再用全量 `test.sh` 验证最终路径。候选规模由 `p`、`nprobe`、`ef` 和 GPU candidate cap 控制: 这些参数越大, recall 越稳, 但 scan、selection 和 cache 压力同步上升。线程规模则通过 1、2、4、8、16、32 等线程 sweep 观察扩展性。这样的规模分析比只改变数据集 N 更贴合 ANN 查询, 因为最终延迟的直接决定因素是“实际访问了多少候选”以及“这些候选如何被线程处理”。

Pthread 与 OpenMP 的跨平台差异也有一致规律。Pthread 固定 worker 更容易复用线程私有 top-k、LUT 和 visited 缓冲, 因此在 IVF 这类候选扫描路径上更接近最小调度开销; OpenMP CPU 的 dynamic schedule 能缓解倒排链长度不均, 但 runtime 需要维护 chunk 分发和隐式 barrier。x86 线程资源更多, 调度成本可以被更多 query 和更高单核性能摊薄; ARM quick 环境线程数较少, 单个 runtime 开销在端到端 latency 中占比更高。因此 OpenMP CPU 在两平台上都能复现 IVF 的 trade-off, 但没有超过手写 Pthread IVF。

11.3 构建时间与查询延迟

索引构建时间不计入单次查询 latency, 但对实际系统部署很重要。Flat 无索引构建, 但查询慢; IVF 需要 KMeans 聚类, 构建时间中等; HNSW 构图更重, 但查询最快。图 9 展示了构建代价和查询延迟之间的关系。

表 13: 最终全量运行路径

平台	算法	线程	参数	latency(μ s)	recall@100	build(s)
x86 Windows	HNSW-StdAsync	32	ef=64	9.316000	0.952683	6.238946
ARM/Kunpeng	HNSW-StdThread	16	ef=64	70.878600	0.952643	43.452011

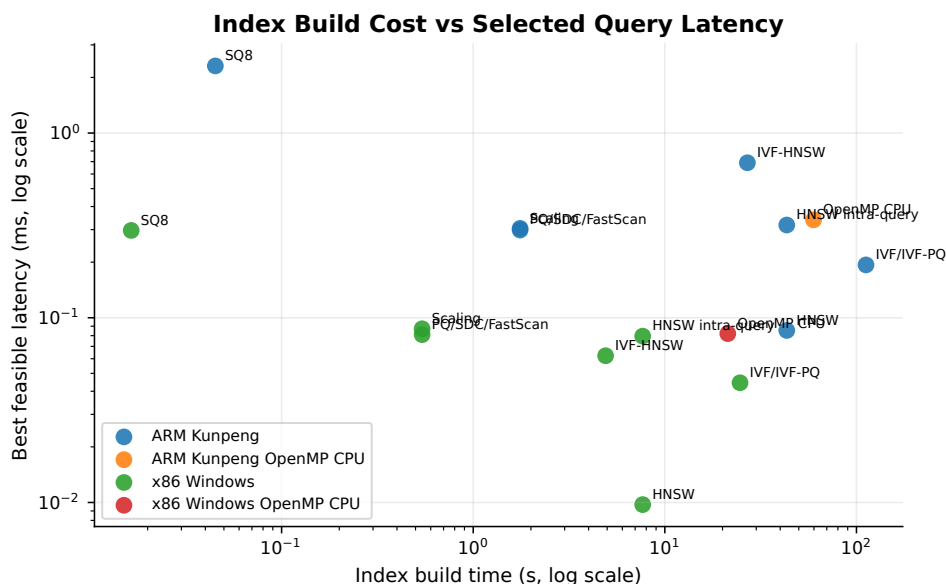


图 9: 索引构建时间与最优查询延迟的关系。HNSW 用更高构建成本换取最低查询延迟。

最终路径与 sweep 结论一致：两个平台都选择 HNSW，但工具和线程数按平台调整。特别是 ARM 结果已经通过全量 `test.sh` 验证，不只是 quick sweep 的估计。

11.4 最终参数选择依据

最终选择 `ef=64` 而不是更大的 `ef=128` 或 `ef=256`，是因为目标函数已经固定为 $\text{recall}@100 \geq 0.95$ 下的最低 latency。以 x86 HNSW 为例，`ef=64` 已达到 $\text{recall}@100=0.954540$ ；继续增大 `ef` 可以提高 recall，但需要扩展更多图节点，平均延迟随之上升。ARM 上也出现相同趋势，因此 `ef=64` 是两个平台都稳定跨过目标线的最低成本点。

线程数也不是越高越好。x86 本地 CPU 的逻辑线程更多，`std::async` 在 32 线程下调度表现最好；ARM/Kunpeng 实机为 8 核环境，16 线程的 `std::thread` 最稳。若强行沿用 x86 的 32 线程 `std::async`，ARM 上会引入更明显的调度和上下文切换开销。最终路径按平台分开选择，是跨平台实验的直接结论，而不是手工偏置。

表 14: 最终路径与主要备选路径比较

平台	方法	latency	recall@100	取舍分析
x86	HNSW final	0.009316 ms	0.952683	全量最终路径，最低延迟。
x86	IVF best	0.044471 ms	0.956950	非图索引最快，但约为 HNSW 的 4.8 倍。
x86	FastScan best	0.081048 ms	0.981710	recall 更高，但延迟不如 IVF/HNSW。
x86	OpenMP CPU IVF	0.082060 ms	0.956950	OpenMP host 最优，慢于手写 Pthread IVF。
ARM	HNSW final	0.070879 ms	0.952643	全量最终路径，优于 ARM quick 中其他算法族。
ARM	IVF best	0.193230 ms	0.956240	稳定但候选扫描多于 HNSW。
ARM	OpenMP CPU IVF	0.338880 ms	0.950767	ARM OpenMP host 最优，dynamic schedule 跨过目标线。
Intel GPU	OpenMP target IVF	0.186502 ms	0.950170	严格卸载到设备，快于 ARM IVF，但慢于 CPU HNSW。

表 14 说明最终路径并不是只看某一个算法的局部结果，而是在所有 recall 高于 0.95 的候选中比较得出。HNSW 的优势来自候选访问量的数量级减少；IVF 的优势是结构简单、参数可解释；FastScan 的优势是单候选计算快；OpenMP target 的优势是规则 batch scan 吞吐高。最终选择 HNSW，是因为它在目标指标下给出了最低 latency。

从进程内并行角度看，最终参数还体现了线程管理代价优化。HNSW 选择 query-level 并行，是因为它让每个线程独立维护候选队列和 visited 状态，避免全局队列加锁；IVF 选择固定 worker 扫描倒排链，是因为候选区间只读，局部 heap 可在线程内复用；OpenMP CPU 选择 dynamic schedule，是因为倒排链长度不均需要一定负载均衡。三者分别对应“线程私有状态”“只读共享索引”“运行时调

度平衡”三类策略。它们的共同原则是：并行化必须保护 top-k 结果一致性，所有线程只能在局部结构中产生候选，最终归并阶段再统一比较距离和 id。

12 Profiling 与体系结构相关优化

12.1 阶段耗时 profiling

为了把“更快”拆解为可解释的瓶颈，本文在 Pthread/OpenMP 阶段记录了候选编码、LUT 构建、候选 scan、top-k selection 和 rerank 的阶段耗时。并不是所有算法都有完整阶段拆分：IVF 与 HNSW 的部分记录只保留 wall time，因为它们的核心瓶颈来自索引遍历和候选访问；PQ/FastScan、OpenMP target 与 SYCL 则能更清晰地区分 scan 和最终 top-k/rerank。GPU IVF 的距离矩阵已经来自原始向量精确距离，因而不存在额外二次精排 kernel，表中的 select/rerank 指扫描后的最终 top-k 重排。

profiling 分成两层。第一层是跨平台都能稳定记录的阶段计时，用 `std::chrono` 或设备事件把 encode/LUT/scan/select/rerank 拆开，直接回答“时间花在哪个阶段”。第二层是体系结构计数器，用 cycles、IPC、L1/LLC miss 等指标解释为什么某阶段难以继续扩展。Windows x86 侧重点在阶段计时和线程 sweep；ARM/Kunpeng 侧结合 SIMD 阶段的 perf counter 观察 cache 行为。两层信息不能互相替代：阶段计时能定位瓶颈阶段，硬件计数器能解释该阶段是算术、分支还是内存受限。

表 15: Pthread/OpenMP 阶段耗时 profiling 摘要

来源	实验	方法	参数	latency(ms)	encode	LUT/fill	scan	select/rerank	主导占比	结论
x86 Pthread	FastScan	b64	p=750,t=32	0.081048	0.000	68.467	786.027	1050.545	51.2% select	查表后 top-k 选择成为瓶颈。
x86 Pthread	PQ-M	M8	p=1500,t=32	0.084262	0.000	15.941	680.023	1015.001	51.5% select	候选多时 selection 压过 LUT 扫描。
x86 OpenMP CPU	PQ-ADC	M16 dynamic	p=500,t=32	0.175798	0.000	157.307	1677.696	2140.931	51.2% select	OpenMP host 仍受 top-k 维护限制。
x86 OpenMP target	IVF	cap3840	nprobe=81	0.186502	27.519	3.831	128.732	19.206	71.8% scan	设备 scan 主导，最终重排仍在 host 端。
x86 SYCL	IVF	n12048	nprobe=84	0.210546	27.471	0.075	178.378	4.622	84.7% scan	补齐计时后仍是 scan-bound，最终重排较轻。
ARM Pthread	FastScan	b128	p=500,t=32	0.298396	0.000	170.414	3845.236	2014.618	60.4% scan	ARM 上 scan 与内存访问占比更高。
ARM Pthread	PQ-M	M8	p=1500,t=32	0.314184	0.000	33.536	2665.096	3070.892	46.1% select	selection 与 scan 接近，候选数仍是核心。
ARM OpenMP CPU	PQ-ADC	M16 dynamic	p=1000,t=8	0.631290	0.000	32.043	3060.700	912.423	66.7% scan	线程较少时扫描阶段更主导。

表 15 支持两个结论。第一，PQ/FastScan 把距离计算压缩后，top-k selection 立刻成为显著瓶颈。x86 FastScan 和 PQ-M 的主导阶段都是 selection，说明后续优化不能只继续微调 LUT 累加，还要减少候选数量或改进 partial top-k。第二，OpenMP target 与 SYCL 的主导阶段是设备 scan，说明 GPU 路径的当前瓶颈主要是规则候选距离矩阵，而不是 host 侧最终重排。补齐 SYCL 的 host 侧计时后可以看到，它与 OpenMP target 一样存在约 27 μ s/query 的 coarse selection 开销；差异在于 SYCL 的 batch candidate fill 被组织得更轻，而 device scan 更重。由于 GPU IVF 已经返回精确候选距离，select/rerank 阶段的优化重点不是增加一次精排，而是减少回传矩阵规模、改进设备端 partial top-k，并考虑把 coarse selection 或候选压缩进一步下沉到设备端。

12.2 cache 与内存访问分析

SIMD 阶段的 ARM perf counter 为 Pthread 阶段提供了体系结构背景。精确 Flat-NEON 的 cycles/query 约为 24.10M，IPC 为 0.88，LLC miss rate 达到 40.11%；FastScan 的 fsadc-m16-p1500-b64 将 cycles/query 降到 9.72M，IPC 提升到 2.14，L1 miss rate 降到 0.60%，但 LLC miss rate 仍有 20.91%。这说明 FastScan 的 layout 改造确实改善了局部访问和指令吞吐，但只要候选数量仍大，末级缓存和内存访问仍会限制多线程扩展。

表 16: SIMD perf counter 对 Pthread 阶段的体系结构背景

kernel	latency(ms)	cycles/query	IPC	L1 miss	LLC miss
flat-neon	9.398667	24.10M	0.88	2.68%	40.11%
pq-m16-p2000	4.340333	12.84M	1.69	0.60%	17.16%
pqsel-m16-p2000	3.584000	10.85M	1.78	0.84%	19.67%
fsadc-m16-p1500-b64	3.124333	9.72M	2.14	0.60%	20.91%

基于这些 profiling 结果，代码实现采用了几类体系结构优化。其一，所有线程的 top-k heap、LUT、visited 标记和 rerank 缓冲都线程私有，避免 false sharing 和锁竞争。其二，FastScan 按 block 重排 code，使一个线程处理完整 block，降低跨线程共享缓存行的概率。其三，IVF 将倒排链表示为连续 id 和 offset 数组，减少链式指针访问；GPU IVF 进一步将候选 id 填成固定矩阵，换取规则 device

kernel。其四，HNSW 使用线程私有 visited 状态和固定容量候选结构，避免每个 query 清空全量数组或在热路径动态分配。

这些优化的共同点是减少“并行之外”的缓存一致性代价。Flat 和 IVF 的主数据是只读 base 向量，多个线程同时读取不会产生写失效；真正容易 false sharing 的是每线程统计量、heap 元数据、候选计数和计时记录。因此实现中让 worker-local 结构按线程分区保存，并尽量避免多个线程交错写同一连续小数组。FastScan 选择完整 block 作为线程分配单位，也是为了让一个 cache line 内的 code 更可能被同一个线程消费。HNSW 的访问更随机，无法像 Flat 那样依赖硬件预取，所以减少 visited 清空和动态分配比继续微调距离核更有效。

profiling 也给出了没有继续扩大线程数的理由。Flat 在 32 线程后主要受内存带宽和 top-k 更新限制；ARM Flat base-split 在 16 线程已经优于 32 线程，说明更多线程带来的缓存竞争和归并成本超过扫描收益。OpenMP CPU IVF 在 ARM 上选择 8 线程，是因为 dynamic schedule 的负载均衡收益已经足够，继续增加线程会让 runtime 和缓存竞争占比上升。HNSW 的单 query 访问节点少，线程数过高时任务分发和队列初始化会成为可见成本。因此，profiling 的结论不是“线程越多越好”，而是每个算法都有由候选规模和内存层次共同决定的饱和点。

12.3 复杂性与线程管理代价

从复杂性看，query-level 并行把 Q 个 query 分给 T 个线程，理想平均工作量约为 $O(Q/T)$ 个 query，但每个 query 的候选量由算法决定。Flat 仍是 $O(ND)$ ，IVF 约为 $O(nlist \cdot D + N \cdot nprobe / nlist \cdot D)$ ，PQ-ADC 将候选距离从 $O(D)$ 改为 $O(M)$ 查表，HNSW 则依赖访问节点数 $V(ef)$ ，近似为 $O(V(ef) \log ef)$ 的队列维护与距离计算。多线程只降低可并行部分的常数，不能替代候选裁剪带来的复杂度下降。

线程管理代价在最终结果中同样明显。Pthread IVF 快于 OpenMP CPU IVF，是因为固定 worker 和线程私有缓冲更接近倒排链扫描的访问模式；OpenMP CPU 的 dynamic schedule 能缓解倒排链负载不均，但带来运行时调度和隐式同步。HNSW 的 `std::async` 在 x86 上最快，而 ARM 上 `std::thread` 更稳，说明标准库工具的表现取决于平台运行时。查询内并行 HNSW 的负优化则进一步说明：当算法依赖全局队列剪枝时，并行粒度过细会增加重复访问和合并成本。

13 综合讨论：算法结构与硬件匹配

13.1 从计算密集到访存密集

SIMD 阶段主要解决的是“单次距离计算如何更快”，而 Pthread 阶段暴露出一个更重要的问题：当单候选距离已经足够快时，系统瓶颈会逐渐从算术计算转向访存、候选选择和线程协作。Flat 搜索最接近 compute-bound，因为每个候选都要计算完整距离，线程扩展和 SIMD 都能直接降低主要计算时间；但 Flat 的候选数固定为 N ，它无法从算法结构上减少访问量。

SQ8、PQ、SDC 和 FastScan 将单候选计算压缩为量化、查表或整数累加。它们在单候选层面比 Flat 更轻，但也更容易进入 memory-bound 状态：查表地址由压缩 code 决定，访问模式不如连续浮点数组规则；多个线程同时扫描 code 时，cache miss 和内存带宽会限制扩展。FastScan 通过更适合 SIMD lane 的布局缓解了这一点，但无法改变“仍要扫描大量候选”的事实。因此，量化路径的优化上限由候选规模和访存模式共同决定。

IVF 和 HNSW 的核心优势是减少候选数量。IVF 使用质心先过滤倒排链，使候选量约为 $N \cdot nprobe / nlist$ ；HNSW 则使用图导航，仅访问与 query 更相关的一小部分节点。二者都从算法结构上减少了后续距离计算和 top-k 维护，因此即使单候选计算未必比 FastScan 更轻，端到端 latency 仍然更低。最终结果显示，HNSW 的候选裁剪能力最强，因此在 CPU 平台上成为最优路径。

13.2 线程粒度的选择原则

线程粒度决定了并行化是降低主要工作量，还是引入额外同步。对 Flat 和 IVF，query-level 并行最稳定，因为每个 query 的 top-k 结构可以独立维护，线程间只共享只读数据。base-split 和倒排链内部分片虽然能降低单 query 延迟，但会引入局部 top-k merge；当候选数不够大或线程数过高时，merge 开销会抵消收益。

对 HNSW, query-level 并行几乎是最自然的选择。标准 HNSW 查询依赖候选队列的动态状态, 当前扩展出的节点会决定下一步访问哪些邻居。Layer 0 多入口、边划分和点划分都试图把这一过程拆开, 但拆开之后每个线程都缺少全局队列视角, 容易产生重复搜索或过早访问不重要节点。实验中这些变体能作为图索引并行探索, 但没有超过标准 query-level HNSW, 说明图搜索算法的同步结构比 Flat/IVF 更难并行。

表 17: 不同并行粒度的收益与代价

并行粒度	适用算法	主要收益	主要代价
query-level	Flat、PQ、IVF、HNSW	线程间状态独立, 几乎无 top-k 归并	需要 batch queries; 单 query latency 不一定下降。
base-split	Flat exact	降低单 query 全量扫描时间	局部 top-p 归并影响 recall 和 latency。
倒排链分片	IVF/IVF-PQ	对长 list 可提升负载均衡	list 长度不均, 候选归并复杂。
OpenMP dynamic query	OpenMP CPU IVF/PQ	缓解 query 或倒排链长度不均	调度器和隐式同步开销高于固定 worker。
LUT/子空间并行	PQ-ADC	LUT 构建阶段规则、易并行	LUT 构建本身占比小, 收益有限。
Layer 0 查询内并行	HNSW	探索单 query 内并行可能性	破坏全局队列剪枝, 重复访问和同步开销高。
GPU batch 并行	OpenMP target/SYCL IVF	大批量候选扫描吞吐高	top-k selection、随机访存和 host-device 协作限制收益。

13.3 近似误差与参数敏感性

ANN 算法的误差来源并不相同。量化类方法的误差主要来自向量表示压缩, SQ8 将每一维映射到整数区间, PQ 将原向量拆成子空间并使用码本近似, FastScan 则进一步把查表布局改造成更适合 SIMD 的形式。它们的 recall 变化通常比较平滑: 候选数 p 、rerank 规模或码本粒度增大时, 返回集合更接近精确 top-k, 但查询阶段要处理更多候选和查表访问。

IVF 的误差主要来自 coarse quantizer 的簇选择。若 query 附近的真实近邻分散在多个倒排链中, 较小的 $nprobe$ 会漏掉部分近邻; 随着 $nprobe$ 增大, 扫描链数和候选数同步上升, recall 与 latency 呈现非常直接的单调关系。HNSW 的误差则来自图导航路径, ef 控制候选队列宽度, 较大的 ef 能让搜索保留更多备选路径, 但也增加邻接点访问、优先队列维护和 cache 压力。

这些参数敏感性决定了消融实验的组织方式。量化类方法需要观察候选数与精排成本, IVF 需要观察 $nprobe$ 对倒排链访问量的影响, HNSW 需要观察 ef 对访问节点数和线程调度的影响, GPU batch IVF 还要额外关注每批 query 的负载均衡、距离矩阵回传和 host 端 top-k 选择成本。只有把误差来源和硬件开销对应起来, 才能解释为什么某些参数在 x86 上有效, 却不一定能迁移到 ARM 或 GPU。

13.4 跨平台差异的解释

x86 与 ARM 的最优算法族相同, 但线程选择不同。IVF 在两个平台上都选择 $n1512$, $nprobe=32$, 说明该参数主要由数据分布和 recall 目标决定; ARM 的绝对延迟更高, 则反映了单核性能、SIMD 宽度和内存层次差异。HNSW 在两个平台上都最优, 但 x86 使用 `std::async` 与 32 线程, ARM 使用 `std::thread` 与 16 线程, 说明单 query 成本较低时, 线程运行时开销会成为决定性因素。

Intel 集显适合规则 batch 扫描, 但 HNSW 的图搜索依赖和随机分支不适合直接卸载。本文因此选择 GPU-friendly 的 IVF。OpenMP target IVF 达到 0.186502 ms/query, 快于 ARM IVF, 但仍慢于 x86 HNSW, 说明设备并行只有在算法结构与硬件执行模型匹配时才会转化为端到端收益。

13.5 对最终方案的工程判断

最终运行路径选择 HNSW query-level 并行, 因为它在两个 CPU 平台上都给出最低 latency, 并且全量查询 recall 稳定超过 0.95。若数据频繁更新, IVF 的构建和查询平衡可能更合适; 若关注 GPU 批量吞吐, OpenMP target/SYCL IVF 仍有继续优化空间; 若强调 SIMD + Pthread 的实现可解释性, FastScan 和 PQ 提供了清晰的单候选优化过程。

因此, 最终结论不是某一种线程接口“最好”, 而是算法结构决定上限, 硬件并行决定能否接近上限。候选数量已经减少后, 再选择低同步开销的并行粒度, 才能得到稳定的跨平台性能。

14 实现组织与结果选择流程

代码结构按算法层、并行层和实验层拆分。距离计算与量化相关逻辑保留在可复用的 SIMD/ANN 基础组件中；Pthread 主线负责 Flat、SQ8、PQ、FastScan、IVF 和 HNSW 的 CPU 多线程实验；OpenMP CPU 单独实现为 host-side benchmark，避免和 OpenMP target 混淆；OpenMP target、SYCL、OpenCL 等异构路径则各自输出独立 CSV。这样的结构使每个结果都能追溯到具体后端和参数，而不是依靠手工记录。

结果选择流程也被固定为脚本化步骤。每次 sweep 都输出完整 CSV，字段至少包含 `experiment`、`method`、`nthreads`、核心参数、`latency\_ms`、`recall@100`、`build\_sec` 和 `notes`。随后脚本先过滤 `recall@100 ≥ 0.95`，再按 `latency` 升序选择最优点。这个流程避免把 `recall` 不达标的低延迟点误判为有效优化，也避免把 `recall` 远高于目标但延迟更大的配置误选为最终路径。HNSW 查询内并行和 GPU candidate cap 调参都使用同一规则，因此负优化结果也能被客观保留和解释。

跨平台部分采用“本地完整、ARM quick、最终全量验证”的策略。x86 本地环境可以完整运行 1000 query sweep，因此承担主要消融、OpenMP target 与 SYCL 实验；ARM/Kunpeng 默认 quick 模式，用较小 query 数筛选参数，再由 `test.sh` 跑全量最终路径。这样既符合 ARM 远端环境的运行约束，也避免把 quick 结果误当最终全量结果。OpenMP CPU 在 ARM 上同样保留 quick 结果，用来和 Pthread 的同平台结果对照；OpenMP target 则集中在本地 Intel GPU 上分析设备端 batch IVF。

并发实现上，线程私有状态是最重要的边界。Flat base-split、PQ-ADC、FastScan、IVF 和 HNSW 都避免多个线程共享 top-k heap、LUT、visited 标记或局部候选数组。共享数据只读，包括 base 向量、PQ codebook、倒排链、HNSW 图结构和 ground truth；写入数据按 query id 或线程 id 分区。这个约束让并行一致性更容易检查，也降低了 false sharing 和锁竞争。对于 OpenMP target，代码进一步避免设备端 STL、动态分配和复杂队列，把 kernel 限制为固定形状的 batch candidate scan，便于普通 -O2 编译和稳定复现。

默认查询路径只保留最优可行配置，而不是把所有探索性后端混入默认运行。x86 默认选择 HNSW-StdAsync，`ef=64`，`threads=32`，ARM 默认选择 HNSW-StdThread，`ef=64`，`threads=16`；二者都在全量查询上跨过 `recall@100` 约束。OpenMP CPU、OpenMP target、SYCL 和查询内并行 HNSW 保留为实验与对比代码。这样的组织方式让默认运行保持最低延迟，同时保留足够的消融证据。

14.1 可靠性检查与平台迁移

实验可靠性首先来自统一的计时边界。构建索引、训练量化器和查询计时被分开记录；最终 `latency` 只统计查询阶段平均时间，`build time` 作为部署代价单独报告。这样可以避免 HNSW、IVF 和 Flat 因构建方式不同而被混在同一个时间指标里比较。GPU 路径也采用相同原则：candidate 填充、设备 scan、host selection 分别计时，分析中不把单个 kernel 时间当作端到端查询延迟。

第二个可靠性检查是统一的 ground truth 与 top-k 统计。所有路径都使用同一份 DEEP100K ground truth 计算 `recall@100`，且最终返回集合统一转成 id 集合后求交集。HNSW 库返回的是堆结构，IVF/PQ 返回的是自定义 top-k 容器，GPU 路径返回的是候选距离矩阵；这些返回形式不同，但评价前都会归一化到相同的 top-100 id 序列。因此不同路径的 `recall` 差异来自算法本身，而不是返回顺序或容器格式差异。

第三个检查是平台相关参数不硬编码为同一组。x86 的最终线程数为 32，ARM/Kunpeng 的最终线程数为 16；OpenMP CPU 在 x86 上选择 32 线程，在 ARM quick 中选择 8 线程。这些选择来自各自平台的 sweep，而不是强行套用同一参数。这样做符合多线程实验的真实情况：算法结构可以迁移，但线程数、调度工具和运行时开销会随硬件、操作系统和编译器变化。

ARM 迁移命令保留 quick 模式，是为了降低远端平台排队和运行时间的不确定性。quick sweep 用于发现可行参数，最终性能仍由 `test.sh` 全量验证。在结果解释中，quick 结果用于消融分析，全量结果用于最终性能对比；二者不混用。x86 本地环境则用于补足大规模消融、OpenMP target 和 SYCL，因为这些实验依赖本机 Intel GPU 与 oneAPI 编译环境。

OpenMP target 的可靠性检查还包括设备执行确认。target 区域设置 mandatory offload，并通过设备端标记确认 kernel 不是在 initial device 上执行。这样可以避免 Windows 环境中最常见的误判：代码看起来使用了 `target pragma`，但实际回退到 host CPU。SYCL 路径也记录 selector、backend、device 名称和编译器版本，使 GPU 结果可以与 OpenMP target 的设备环境对应起来。

最后，报告保留负优化结果而不是删除它们。HNSW 查询内并行、IVF-HNSW、OpenMP CPU 与 GPU IVF 都没有超过 CPU HNSW，但它们分别揭示了图索引查询内并行的同步难点、OpenMP host 与 Pthread 的调度差异，以及 GPU-friendly batch scan 受 top-k 和候选裁剪限制的原因。保留这些结果能防止报告变成只展示最终最快点的单点汇报，也能说明最终选择 HNSW query-level parallel 是经过消融筛选得到的。

15 优化过程总结

本阶段优化过程可以按四个阶段理解。第一阶段是并行精确搜索，Flat-SIMD + Pthread/OpenMP CPU 验证了多线程能降低精确搜索延迟，但仍要访问全部 base，因此主要作为 recall 上界和 speedup 基线。第二阶段是压缩单候选计算，SQ8、PQ、SDC 和 FastScan 将浮点距离转为量化、查表和整数累加；FastScan 在 x86 上达到 0.081048 ms/query，但由于候选规模仍大，最终慢于 IVF 和 HNSW。

第三阶段是减少候选数量。IVF 通过 nprobe 控制倒排链扫描范围，是最快的非图索引；OpenMP CPU IVF 复现了同一 trade-off，但 dynamic schedule 的调度成本使其慢于手写 Pthread IVF。HNSW 进一步利用图导航避免大量无关候选，成为最终最优路径。第四阶段是匹配硬件并行模型：CPU 上 query-level parallel 最稳定，HNSW 查询内并行因破坏图搜索依赖而负优化；GPU 上必须使用 batch IVF 这类规则并行结构来发挥吞吐优势。

16 结论

本文围绕 SIMD 基础上的 ANN 多线程与加速器并行展开实验。所有结果统一使用 recall@100，并以 $\text{recall@100} \geq 0.95$ 下最小 latency 为唯一最优准则。Pthread 与 OpenMP CPU 的双平台结果表明，IVF dynamic schedule 是稳定的 host OpenMP 路径，但手写 Pthread IVF 更快；x86 和 ARM 的最终最优均为 HNSW query-level 并行，说明图索引候选裁剪比单纯增加线程数更关键。PQ/FastScan 能降低单候选成本，但受候选规模、top-k selection 和访存限制。OpenMP target 在 Intel GPU 上执行 batch IVF，能够利用规则候选扫描吞吐，但候选规模、host-device 协作和 host 端 selection 使其在本实验规模下仍慢于 CPU HNSW。

生成式 AI 辅助学习附录

本附录整理生成式 AI 辅助学习问答。

问答 1: Pthread 在 ANN 中适合承担什么并行粒度

用户问题：对于 ANN 查询，Pthread 更适合 query-level 并行、base-split 并行，还是倒排链内部并行？

AI 回答：Pthread 的优势在于显式控制 worker 生命周期、工作划分和线程私有缓冲，因此最适合放在能保持状态独立的粒度上。query-level 并行通常是最稳的选择：每个线程处理一段 query，base 向量、IVF 倒排链、PQ codebook 或 HNSW 图结构只读共享，top-k heap、visited 标记、LUT 和 rerank 缓冲都放在线程私有内存中。这种粒度几乎不需要锁，最终结果可以按 query id 直接写回。

base-split 并行适合精确 Flat 这类候选量极大的路径。它把同一个 query 的 base 区间拆给多个线程，每个线程产生局部 top-k 或 top-p，最后再归并。它能降低单个 query 的 wall time，但会引入局部结果归并、同步等待和更多 top-k 写入。IVF 的倒排链内部并行介于二者之间：当 nprobe 较大、候选数较多时可以切分扫描区间；当 nprobe 较小或 HNSW 已经把候选访问量压低时，继续拆分单 query 往往会让同步和归并成本占比升高。因此本实验中最终 CPU 路径选择 HNSW query-level 并行，而 Flat/base-split 和 IVF 内部切分主要用于消融。

问答 2: Pthread worker 为什么要复用线程私有缓冲

用户问题：Pthread ANN 实现中，为什么要给每个 worker 预分配局部 top-k、LUT、visited 和候选数组？

AI 回答：ANN 查询的热路径非常短，尤其是 HNSW 和 IVF 这类索引裁剪后，单个 query 实际访问的候选数已经远小于 Flat 全量扫描。此时如果每个 query 都动态申请 heap、visited 数组或 PQ lookup table，内存分配器、初始化和 cache 污染会变成显著开销。Pthread worker 固定后，可以在线程启动时分配一次局部缓冲，在多个 query 之间反复覆盖使用，从而避免热循环中的 malloc/free 或容器扩容。

线程私有缓冲还解决了并发一致性问题。top-k heap 如果被多个线程共享，就需要锁或原子操作；PQ LUT 如果共享，query 切换时会出现数据竞争；HNSW visited 标记如果全局共享，则不同 query 的访问状态会互相污染。将这些状态放入 worker-local storage 后，共享数据只剩只读索引和向量数组，写入数据按 query id 或线程 id 分区，CPU cache coherence 的压力会明显降低。对 Pthread 来说，性能并不只来自多开线程，而是来自把线程边界和算法状态边界对齐。

问答 3: OpenMP CPU 与 Pthread 的性能差异来自哪里

用户问题：OpenMP CPU 和 Pthread 最终都使用 CPU 线程，为什么在 IVF 或 Flat 上可能表现不同？

AI 回答：二者底层都可能映射到操作系统线程，但控制粒度不同。Pthread 版本通常显式创建固定 worker，并把 query 区间、base 区间或倒排链区间交给 worker 长期执行，线程私有缓冲也能和 worker 生命周期绑定。OpenMP CPU 则通过 `\#pragma omp parallel for`、`schedule(static)` 或 `schedule(dynamic)` 描述循环并行，线程池、chunk 切分、隐式 barrier 和负载均衡由 runtime 管理。

当每个迭代工作量较大且规则时，OpenMP 的 runtime 开销容易被摊薄，代码也更简洁；但 ANN 中很多优化路径的单 query 工作量已经被索引裁剪得很小，调度和同步就会进入端到端延迟。以 IVF 为例，dynamic schedule 可以缓解倒排链长度不均，但每个 chunk 的获取和隐式同步都要付出成本；Pthread 固定分块可能负载均衡略弱，却减少了 runtime 调度层。最终哪一个更快，取决于候选扫描成本、query 数量、线程数、倒排链不均衡程度和 top-k 合并成本，而不是简单由“是否 OpenMP”决定。

问答 4: OpenMP schedule 对 ANN 有什么影响

用户问题：在 OpenMP CPU 路径中，static、dynamic 和 chunk size 应该如何理解？

AI 回答： `schedule(static)` 在进入并行循环时把迭代空间固定分给各线程，调度成本低、cache 行为更可预测，适合每个 query 候选量接近的情况，例如 Flat query-level 精确扫描。它的问题是遇到负载不均衡时可能出现长尾线程：某些线程处理的 query 需要扫描更多倒排链或更多候选，其他线程已经结束但必须在隐式 barrier 等待。

`schedule(dynamic)` 将迭代按 chunk 动态分发给空闲线程，适合 IVF 这类不同 query 的 fine scan 成本不均衡的算法。它能减少长尾等待，但每个 chunk 都需要 runtime 维护队列和分配状态。chunk size 太小会增加调度次数，chunk size 太大又退化成 static 式长尾。因此 OpenMP ANN 的调参不只是线程数，还包括 schedule 与 chunk 粒度。对 $\text{recall}@100 \geq 0.95$ 附近的 IVF，候选数被 `nprobe` 控制在中等规模，dynamic schedule 的负载均衡收益和 runtime 成本会同时存在，这也是它能成为 OpenMP CPU 最优路径但仍慢于手写 Pthread IVF 的主要原因。

问答 5：局部 top-k 合并为什么是并行 ANN 的关键代价

用户问题： 多线程距离扫描看起来很容易并行，为什么 top-k 合并仍然会影响整体延迟？

AI 回答： ANN 查询不仅要计算距离，还要维护最小的一组候选。对 query-level 并行，每个 query 只由一个线程处理，top-k 完全局部化，合并成本最低。对 base-split 或倒排链内部分片并行，同一个 query 的候选被多个线程扫描，每个线程只能得到局部 top-k，最后必须把这些局部结果归并成全局 top-100。这个归并过程包含比较、分支、去重和结果写回，难以像距离计算一样被 SIMD 或 GPU 完全规则化。

如果使用共享 top-k heap，多线程会围绕 heap 插入和阈值更新产生锁竞争；如果使用线程私有 heap，最后的 merge 虽然避免锁，但会增加一次额外选择。候选数越多，局部 top-k 维护越重；线程数越多，局部结果数量越多，最终归并也越重。这个问题解释了为什么 Flat base-split 能降低扫描时间但未必成为最终最优，也解释了 GPU IVF 中 host 端 selection 仍然会限制端到端延迟。高性能 ANN 的核心不是让所有距离计算都并行，而是让候选生成、距离 scan 和 top-k selection 三者的代价匹配。

问答 6：false sharing 在 Pthread ANN 中如何出现

用户问题： Pthread 版本中为什么要关注 false sharing，它在 ANN 代码里通常发生在哪里？

AI 回答： false sharing 指多个线程写入不同变量，但这些变量落在同一个 cache line 上，导致 cache coherence 协议频繁使 cache line 失效。ANN 查询中常见风险包括：每个线程的统计计数器相邻存放、局部 top-k 元数据连续存放、每个线程的输出游标或耗时记录放在同一个小数组中。虽然逻辑上没有数据竞争，但硬件会以 cache line 为单位维护一致性，多个核心反复写同一 cache line 会造成额外延迟。

Pthread 实现通常通过两种方式降低 false sharing。第一，线程私有结构按 cache line 对齐或加入 padding，使 worker 的 heap、计数器和临时状态不落在同一 cache line。第二，结果写入按 query id 分区，避免多个线程交错写相邻位置。对 Flat 和 IVF 这种扫描密集路径，false sharing 会破坏原本连续读 base 的带宽优势；对 HNSW 这种单 query 成本很低的路径，少量 cache coherence 开销也会被放大。因此多线程 ANN 的内存布局和线程调度同样重要。

问答 7：Pthread 中的 barrier 和 join 应该如何放置

用户问题： Pthread ANN benchmark 中，为什么不能在每个 query 后做线程同步？

AI 回答： 每个 query 后同步会把并行粒度压得过细。假设有 Q 个 query 和 T 个线程，如果每个 query 都触发一次 barrier，那么系统会执行 Q 次全线程同步；而 query-level 分块只需要在线程启动和结束时同步。对 HNSW、IVF 这类单 query 延迟已经很低的算法，barrier 的固定成本可能接近甚至超过搜索本身。

更合理的做法是让 worker 获取一段连续 query 或动态 query 块，在块内无同步地循环执行。主线程只在所有 worker 结束后 `pthread\_join`，然后统一统计 recall 和 latency。若需要动态负载均衡，也应使用粗粒度任务队列或原子 query cursor，而不是在每个 query 设置全局 barrier。OpenMP 中的隐式 barrier 也有类似问题，因此在只需要并行循环结果的场景，可以考虑 `nowait` 或合并多个循环，减少不必要的同步边界。

问答 8: HNSW 查询内并行为什么容易负优化

用户问题: 为什么 HNSW 的边划分、Layer 0 点划分或多入口点并行很难超过 query-level 并行?

AI 回答: HNSW 的低延迟来自图搜索的剪枝, 而剪枝依赖一个全局候选队列。标准查询每次弹出当前最有希望的节点, 访问其邻居, 并用结果队列中的最远距离控制是否继续扩展。这个过程是强顺序依赖: 当前扩展哪个节点会改变后续队列状态, 也会改变哪些邻居被剪掉。把一个 query 内部拆成多个线程后, 各线程看到的队列状态往往不是最新全局状态, 容易重复访问节点, 或者扩展一些本来会被全局队列剪掉的候选。

边划分并行只加速“当前节点邻居距离计算”这一小段, 但队列 push/pop、visited 判断和终止条件仍然串行; Layer 0 点划分让线程各自维护局部搜索, 会丢失全局最优扩展顺序; 多入口点并行能增加搜索多样性, 但也会扩大访问范围并增加最终归并成本。HNSW 单 query 访问节点数本来就不大, 这些额外同步、重复访问和合并开销很容易抵消距离计算收益。因此 query-level 并行更符合 HNSW 的结构: 多个 query 之间独立, 图索引只读共享, 线程间不破坏单 query 的全局候选队列。

问答 9: IVF+HNSW 嵌套结构的并行收益和风险是什么

用户问题: 多个 HNSW 或 IVF+HNSW 嵌套结构为什么适合做多线程探索, 但不一定最快?

AI 回答: IVF+HNSW 的思路是先用 coarse quantizer 把 base 向量分到多个簇, 再在每个簇内部建立较小的 HNSW。查询时先选择若干 IVF 簇, 然后让多个线程分别搜索不同子图, 最后归并 top-100。这个结构天然具有分区并行性: 不同子图之间只读且相互独立, 线程不需要竞争同一个 HNSW 候选队列。

风险在于两层近似误差会叠加。IVF 的 nprobe 如果没有覆盖真实近邻所在簇, 后续 HNSW 再强也无法找回; 每个子图 HNSW 较小, 图连通性和导航质量也可能变差。并行上, 多个子图搜索会带来更多入口、更多局部 top-k 和一次全局归并, 访问候选数未必少于单个全局 HNSW。它适合展示“粗粒度分区并行”的可行性, 也适合大规模数据分片场景; 但在 DEEP100K 这类规模下, 单个高质量 HNSW 的图导航通常更直接, 候选裁剪更强, 端到端延迟更低。

问答 10: OpenMP target 和 OpenMP CPU 的数据模型有什么根本差别

用户问题: 为什么 OpenMP target 不能简单看作把 OpenMP CPU 循环搬到 GPU 上?

AI 回答: OpenMP CPU 的并行区运行在共享主机内存中, 线程可以直接访问同一地址空间; OpenMP target 则涉及 host-device 数据映射, 变量需要通过 map(to:), map(from:) 或 target data 区域进入设备地址空间。即使语法上仍然是 OpenMP, 性能模型已经从 CPU 线程调度变成了设备 kernel 启动、数据传输、设备端内存访问和 host-device 同步。

ANN 中很多结构不适合直接 offload。例如 HNSW 的优先队列、动态 visited 集合、STL 容器和不规则邻接访问, 在 GPU 上会造成分支发散和随机访存。OpenMP target 更适合被组织成固定形状的 batch IVF scan: host 端生成候选 id 矩阵, device 端对 query $\times$ candidate 二维空间做规则距离计算, host 端再执行 top-k selection。这样做牺牲了一部分端到端简洁性, 但让设备端 kernel 足够规则, 也能明确区分 candidate 填充、device scan、回传和 selection 的耗时。

问答 11: OpenMP target 中 candidate cap 的作用是什么

用户问题: GPU IVF 为什么需要 candidate cap, 它如何影响 recall 和 latency?

AI 回答: IVF 倒排链长度天然不均衡, 同一个 nprobe 下, 不同 query 访问的候选数可能差异很大。GPU 更擅长规则矩阵式工作, 因此 OpenMP target 路径通常把每个 query 的候选填入固定长度数组, 长度上限就是 candidate cap。这样 device kernel 可以遍历 $q_count \times candidate_cap$ 个 slot, 每个 slot 计算一个候选距离或写入无效距离, 线程映射和输出矩阵都很规整。

candidate cap 同时是算法参数和系统参数。cap 太小会截断部分倒排链候选, 可能漏掉真实近邻, 导致 recall 降低; cap 太大则会增加无效 slot、设备端扫描量、距离矩阵回传规模和 host selection 成本。它和 nprobe 存在耦合: nprobe 决定访问多少倒排链, cap 决定最多保留多少候选。OpenMP target 的最优点通常出现在 recall 刚好稳定越过阈值的位置, 而不是 cap 最大的位置, 因为后者会把 GPU 吞吐浪费在过多候选和回传数据上。

问答 12: 为什么 GPU IVF 仍可能慢于 CPU HNSW

用户问题: OpenMP target 的设备端 scan 有并行吞吐, 为什么端到端延迟仍可能高于 CPU HNSW?

AI 回答: GPU 优势主要体现在规则、大批量、算术密集的 scan 上; HNSW 优势来自算法层面的候选裁剪。若 HNSW 只访问很少节点, 而 IVF GPU 为了达到同样 recall 需要扫描几千个候选, 那么即使单候选距离计算更快, 候选总量也可能压倒吞吐优势。换言之, GPU 加速的是“每个候选的处理速度”, HNSW 减少的是“需要处理的候选数量”, 两者不在同一层面。

此外, OpenMP target 的端到端时间不等于 kernel 时间。host 需要完成 coarse quantizer、candidate 填充和数据映射; device 计算距离后, 还要把距离矩阵回传给 host; top-100 selection 仍含有分支和不规则写入。Intel 集显还与 CPU 共享内存层次和功耗预算, 设备 scan 的吞吐并不等同于独立高带宽 GPU。GPU IVF 的优化方向应是减少候选矩阵规模、下沉 partial top-k、降低回传数据量, 而不是单纯扩大 batch 或 candidate cap。

问答 13: OpenMP target 与 SYCL 在 Intel GPU 上如何比较

用户问题: OpenMP target 和 oneAPI/SYCL 都能使用 Intel GPU, 比较时应该看哪些技术点?

AI 回答: OpenMP target 更适合从 CPU 循环渐进迁移: 用 pragma 标注 target teams distribute parallel for, 再通过 map 子句描述数据移动。它的优势是改动小、和 OpenMP CPU 语义接近, 适合展示“同一算法从 host 线程到 device kernel”的迁移过程。缺点是 kernel 组织、内存层级和异步执行的显式控制较弱, 复杂优化依赖编译器和 runtime。

SYCL 更接近 C++ 异构编程模型, 使用 queue、buffer/USM、handler 和 kernel lambda 描述设备计算, 可以更明确地管理事件依赖、内存生命周期和 kernel 粒度。比较二者时不应只看总 latency, 还应拆分 build、candidate fill、device scan、selection 和数据移动; 也要记录 backend、device name、编译器版本和优化等级。若两者瓶颈都集中在候选扫描或 host selection, 说明问题主要来自算法和数据组织; 若差异集中在 kernel 时间或数据移动, 则需要进一步分析编译器向量化、内存 coalescing 和 runtime 调度。

问答 14: Pthread/OpenMP 的 profiling 应关注哪些硬件指标

用户问题: 多线程 ANN profiling 中, 除了 latency, 还应关注哪些 CPU 与内存指标?

AI 回答: 首先应拆分阶段耗时, 包括 coarse selection、LUT 构建、candidate scan、top-k selection、rerank、HNSW graph traversal 和线程同步。单个总时间只能告诉哪个配置快, 不能解释为什么快。其次应关注 IPC、cycles/query、L1/L2/LLC miss、分支失误、内存带宽和上下文切换。Flat 通常更接近带宽受限; PQ/FastScan 更容易看到查表布局和 cache 命中率变化; HNSW 则更容易受随机访存、分支和队列维护影响。

对 Pthread/OpenMP 对比, 还应特别观察负载均衡和同步。OpenMP dynamic schedule 如果降低了长尾等待, 但增加了 runtime 调度开销, 需要从线程时间分布和 barrier 等待中解释; Pthread 固定分块如果某些线程运行明显更久, 则说明分块与候选分布不匹配。对于 ARM 和 x86, SIMD 宽度、cache 层次和线程 runtime 不同, 同一算法可能表现出不同瓶颈。profiling 的目标不是罗列指标, 而是把指标映射回具体代码结构: 是候选太多、top-k 太重、cache miss 太高, 还是同步粒度过细。

问答 15: 跨平台 Pthread/OpenMP 结果为什么不能直接套同一线程数

用户问题: 为什么 x86 上最优线程数不能直接迁移到 ARM/Kunpeng?

AI 回答: 线程数选择受硬件核心数、SMT/超线程、cache 容量、内存带宽、操作系统调度和线程 runtime 共同影响。x86 笔记本平台可能拥有更多逻辑线程和更高单核频率, std::async 或 Pthread worker 在 32 线程附近表现较好; ARM/Kunpeng 的核心结构、NEON 吞吐、内存层次和系统负载不同, 过高线程数可能带来上下文切换、cache 竞争和内存带宽饱和。

ANN 算法还会放大这种差异。Flat 每个 query 工作量大, 更多线程更容易被利用; HNSW 单 query 已经很快, 线程管理成本占比更高; IVF 的倒排链长度不均又会影响 schedule 选择。因此跨平台比较应先保持算法参数可解释, 例如相同的 nlist/nprobe 或 ef, 再分别 sweep 线程数和调度工具。最终结果可以呈现“算法结构可迁移、线程运行时选择不可简单迁移”的结论, 这比强行使用同一线程数更符合多线程性能分析。

问答 16: Pthread、OpenMP CPU 与 OpenMP target 如何形成完整实验链

用户问题: 从实验设计角度看, Pthread、OpenMP CPU 和 OpenMP target 应如何互相支撑?

AI 回答: Pthread 路径提供最细粒度的 CPU 多线程控制, 可以展示 worker 复用、线程私有缓冲、局部 top-k、HNSW visited 和显式归并等工程细节。OpenMP CPU 路径提供共享内存并行模型对照, 可以在相同 Flat/PQ/IVF 算法上比较 *static/dynamic* schedule、隐式 barrier 和 runtime 调度成本。二者共同回答“CPU 多线程如何影响 ANN 查询”的问题。

OpenMP target 则把问题推进到异构设备: 它不应重复 Pthread 的 HNSW query-level 逻辑, 而应选择 GPU-friendly 的 batch IVF scan, 分析 host-device 数据组织、device kernel、回传和 selection。这样三条线形成清晰层次: Pthread 说明手写线程和算法状态管理, OpenMP CPU 说明 pragma 并行与调度代价, OpenMP target 说明加速器卸载的收益边界。最终再用统一的 *recall@100* 和 *latency* 筛选规则比较, 才能把“线程工具差异”“算法候选裁剪”和“硬件执行模型”分开解释。

参考文献

- [1] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE TPAMI*, 2020.
- [2] Z. Fu, W. Xiang, N. Wang, and D. Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. *PVLDB*, 2019.
- [3] H. Jegou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *IEEE TPAMI*, 2011.
- [4] J. Johnson, M. Douze, and H. Jegou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.